```
    }

    return mw
}
```

*Listing 13-3: Creating a sustained multiwriter (*writer.go*)*

First, you instantiate a new `*sustainedMultiWriter`, initialize its writers slice ❶, and cap it to the expected length of writers. You then loop through the given writers and append them to the slice ❹. If a given writer is itself a `*sustainedMultiWriter` ❷, you instead append its writers ❸. Finally, you return the pointer to the initialized `sustainedMultiWriter`.

You can now put your sustained multiwriter to good use in Listing 13-4.

```
package ch13

import (
    "bytes"
    "fmt"
    "log"
    "os"
)

func Example_logMultiWriter() {
    logFile := new(bytes.Buffer)
    w := ❶SustainedMultiWriter(os.Stdout, logFile)
    l := log.New(w, "example: ", ❷log.Lshortfile|log.Lmsgprefix)

    fmt.Println("standard output:")
    l.Print("Canada is south of Detroit")

    fmt.Print("\nlog file contents:\n", logFile.String())

    // Output:
    // standard output:
    // log_test.go:24: example: Canada is south of Detroit
    //
    // log file contents:
    // log_test.go:24: example: Canada is south of Detroit
}
```

*Listing 13-4: Logging simultaneously to a log file and standard output (*log_test.go*)*

You create a new sustained multiwriter ❶, writing to standard output, and a `bytes.Buffer` meant to act as a log file in this example. Next, you create a new logger using your sustained multiwriter, the prefix `example:`, and two flags ❷ to modify the logger's behavior. The addition of the `log.Lmsgprefix` flag (first available in Go 1.14) tells the logger to locate the prefix just before the log message. You can see the effect this has on the log entries in the example output. When you run this example, you see that the logger writes the log entry to the sustained multiwriter, which in turn writes the log entry to both standard output and the log file.

Leveled Log Entries

I wrote earlier in the chapter that verbose logging may be inefficient in production and can overwhelm you with the sheer number of log entries as your service scales up. One way to avoid this is by instituting *logging levels*, which assign a priority to each kind of event, enabling you to always log high-priority errors but conditionally log low-priority entries more suited for debugging and development purposes. For example, you'd always want to know if your service is unable to connect to its database, but you may care to log only details about individual connections while in development or when diagnosing a failure.

I recommend you create just a few log levels to begin with. In my experience, you can address most use cases with just an *error* level and a *debug* level, maybe even an *info* level on occasion. Error log entries should accompany some sort of alert, since these entries indicate a condition that needs your attention. Info log entries typically log non-error information. For example, it may be appropriate for your use case to log a successful database connection or to add a log entry when a listener is ready for incoming connections on a network socket. Debug log entries should be verbose and serve to help you diagnose failures, as well as aid development by helping you reason about the workflow.

Go's ecosystem offers several logging packages, most of which support numerous log levels. Although Go's `log` package does not have inherent support for leveled log entries, you can add similar functionality by creating separate loggers for each log level you need. Listing 13-5 does this: it writes log entries to a log file, but it also writes debug logs to standard output.

```
--snip--

func Example_logLevels() {
    lDebug := log.New(os.Stdout, ❶"DEBUG: ", log.Lshortfile)
    logFile := new(bytes.Buffer)
    w := SustainedMultiWriter(logFile, ❷lDebug.Writer())
    lError := log.New(w, ❸"ERROR: ", log.Lshortfile)

    fmt.Println("standard output:")
    lError.Print("cannot communicate with the database")
    lDebug.Print("you cannot hum while holding your nose")

    fmt.Print("\nlog file contents:\n", logFile.String())

    // Output:
    // standard output:
    // ERROR: log_test.go:43: cannot communicate with the database
    // DEBUG: log_test.go:44: you cannot hum while holding your nose
    //
    // log file contents:
    // ERROR: log_test.go:43: cannot communicate with the database
}
```

*Listing 13-5: Writing debug entries to standard output and errors to both the log file and standard output (*log_test.go*)*

First, you create a debug logger that writes to standard output and uses the `DEBUG:` prefix ❶. Next, you create a `*bytes.Buffer` to masquerade as a log file for this example and instantiate a sustained multiwriter. The sustained multiwriter writes to both the log file and the debug logger's `io.Writer` ❷. Then, you create an error logger that writes to the sustained multiwriter by using the prefix `ERROR:` ❸ to differentiate its log entries from the debug logger. Finally, you use each logger and verify that they output what you expect. Standard output should display log entries from both loggers, whereas the log file should contain only error log entries.

As an exercise, figure out how to make the debug logger conditional without wrapping its `Print` call in a conditional. If you need a hint, you'll find a suitable writer in the `io/ioutil` package that will let you discard its output.

This section is meant to demonstrate additional uses of the `log` package beyond what you've used so far in this book. Although it's possible to use this technique to log at different levels, you'd be better served by a logger with inherent support for log levels, like the Zap logger described in the next section.

Structured Logging

The log entries made by the code you've written so far are meant for human consumption. They are easy for you to read, since each log entry is little more than a message. This means that finding log lines relevant to an issue involves liberal use of the `grep` command or, at worst, manually skimming log entries. But this could become more challenging if the number of log entries increases. You may find yourself looking for a needle in a haystack. Remember, logging is useful only if you can quickly find the information you need.

A common approach to solving this problem is to add metadata to your log entries and then parse the metadata with software to help you organize them. This type of logging is called *structured logging*. Creating structured log entries involves adding key-value pairs to each log entry. In these, you may include the time at which you logged the entry, the part of your application that made the log entry, the log level, the hostname or IP address of the node that created the log entry, and other bits of metadata that you can use for indexing and filtering. Most structured loggers encode log entries as JSON before writing them to log files or shipping them to centralized logging servers. Structured logging makes the whole process of collecting logs in a centralized server easy, since the additional metadata associated with each log entry allows the server to organize and collate log entries across services. Once they're indexed, you can query the log server for specific log entries to better find timely answers to your questions.

Using the Zap Logger

Discussing specific centralized logging solutions is beyond the scope of this book. If you're interested in learning more, I suggest you initially investigate

Elasticsearch or Apache Solr. Instead, this section focuses on implementing the logger itself. You'll use the Zap logger from Uber, found at *https://pkg.go.dev/go.uber.org/zap/*, which allows you to integrate log file rotation.

Log file rotation is the process of closing the current log file, renaming it, and then opening a new log file after the current log file reaches a specific age or size threshold. Rotating log files is a good practice to prevent them from filling up your available hard drive space. Plus, searching through smaller, date-delimited log files is more efficient than searching through a single, monolithic log file. For example, you may want to rotate your log files every week and keep only eight weeks' worth of rotated log files. If you wanted to look at log entries for an event that occurred last week, you could limit your search to a single log file. Also, you can compress the rotated log files to further save hard drive space.

I've used other structured loggers on large projects, and in my experience, Zap causes the least overhead; I can use it in busy bits of code without a noticeable performance hit, unlike other heavyweight structured loggers. But your mileage may vary, so I encourage you to find what works best for you. You can apply the structured logging principles and log file management techniques described here to other structured loggers.

The Zap logger includes `zap.Core` and its options. The `zap.Core` has three components: a log-level threshold, an output, and an encoder. The *log-level threshold* sets the minimum log level that Zap will log; Zap will simply ignore any log entry below that level, allowing you to leave debug logging in your code and configure Zap to conditionally ignore it. Zap's *output* is a `zapcore.WriteSyncer`, which is an `io.Writer` with an additional `Sync` method. Zap can write log entries to any object that implements this interface. And the *encoder* can encode the log entry before writing it to the output.

Writing the Encoder

Although Zap provides a few helper functions, such as `zap.NewProduction` or `zap.NewDevelopment`, to quickly create production and development loggers, you'll create one from scratch, starting with the encoder configuration in Listing 13-6.

```
package ch13

import (
    "bytes"
    "fmt"
    "io/ioutil"
    "log"
    "os"
    "path/filepath"
    "runtime"
    "testing"
    "time"

    "go.uber.org/zap"
    "go.uber.org/zap/zapcore"
    "gopkg.in/fsnotify.v1"
```

```
    "gopkg.in/natefinch/lumberjack.v2"
)

var encoderCfg = zapcore.EncoderConfig{
    MessageKey: ❶"msg",
    NameKey:    ❷"name",

    LevelKey:    "level",
    EncodeLevel: ❸zapcore.LowercaseLevelEncoder,

    CallerKey:    "caller",
    EncodeCaller: ❹zapcore.ShortCallerEncoder,

 ❺ // TimeKey:   "time",
    // EncodeTime: zapcore.ISO8601TimeEncoder,
}
```

*Listing 13-6: The encoder configuration for your Zap logger (*zap_test.go*)*

The encoder configuration is independent of the encoder itself in that you can use the same encoder configuration no matter whether you're passing it to a JSON encoder or a console encoder. The encoder will use your configuration to dictate its output format. Here, your encoder configuration dictates that the encoder use the key `msg` ❶ for the log message and the key `name` ❷ for the logger's name in the log entry. Likewise, the encoder configuration tells the encoder to use the key `level` for the logging level and encode the level name using all lowercase characters ❸. If the logger is configured to add caller details, you want the encoder to associate these details with the `caller` key and encode the details in an abbreviated format ❹.

Since you need to keep the output of the following examples consistent, you'll omit the `time` key ❺ so it won't show up in the output. In practice, you'd want to uncomment these two fields.

Creating the Logger and Its Options

Now that you've defined the encoder configuration, let's use it in Listing 13-7 by instantiating a Zap logger.

```
--snip--

func Example_zapJSON() {
    zl := zap.New(
     ❶ zapcore.NewCore(
         ❷ zapcore.NewJSONEncoder(encoderCfg),
         ❸ zapcore.Lock(os.Stdout),
         ❹ zapcore.DebugLevel,
        ),
     ❺ zap.AddCaller(),
        zap.Fields(
         ❻ zap.String("version", runtime.Version()),
        ),
    )
    defer func() { _ = ❼zl.Sync() }()
```

```
    example := ❽zl.Named("example")
    example.Debug("test debug message")
    example.Info("test info message")

    // Output:
 ❾ // {"level":"debug","name":"example","caller":"ch13/zap_test.go:49",
"msg":"test debug message","version":"❿go1.15.5"}
    // {"level":"info","name":"example","caller":"ch13/zap_test.go:50",
"msg":"test info message","version":"go1.15.5"}
}
```

Listing 13-7: Instantiating a new logger from the encoder configuration and logging to JSON (zap_test.go)

The `zap.New` function accepts a `zap.Core` ❶ and zero or more `zap.Options`. In this example, you're passing the `zap.AddCaller` option ❺, which instructs the logger to include the caller information in each log entry, and a field ❻ named `version` that inserts the runtime version in each log entry.

The `zap.Core` consists of a JSON encoder using your encoder configuration ❷, a `zapcore.WriteSyncer` ❸, and the logging threshold ❹. If the `zapcore.WriteSyncer` isn't safe for concurrent use, you can use `zapcore.Lock` to make it concurrency safe, as in this example.

The Zap logger includes seven log levels, in increasing severity: `DebugLevel`, `InfoLevel`, `WarnLevel`, `ErrorLevel`, `DPanicLevel`, `PanicLevel`, and `FatalLevel`. The `InfoLevel` is the default. `DPanicLevel` and `PanicLevel` entries will cause Zap to log the entry and then panic. An entry logged at the `FatalLevel` will cause Zap to call `os.Exit(1)` after writing the log entry. Since your logger is using `DebugLevel`, it will log all entries.

I recommend you restrict the use of `DPanicLevel` and `PanicLevel` to development and `FatalLevel` to production, and only then for catastrophic startup errors, such as a failure to connect to the database. Otherwise, you're asking for trouble. As mentioned earlier, you can get a lot of mileage out of `DebugLevel`, `ErrorLevel`, and on occasion, `InfoLevel`.

Before you start using the logger, you want to make sure you defer a call to its `Sync` method ❼ to ensure all buffered data is written to the output.

You can also assign the logger a name by calling its `Named` method ❽ and using the returned logger. By default, a logger has no name. A named logger will include a name key in the log entry, provided you defined one in the encoder configuration.

The log entries ❾ now include metadata around the log message, so much so that the log line output exceeds the width of this book. It's also important to mention that the Go version ❿ in the example output is dependent on the version of Go you're using to test this example. Although you're encoding each log entry in JSON, you can still read the additional metadata you're including in the logs. You could ingest this JSON into something like Elasticsearch and run queries on it, letting Elasticsearch do the heavy lifting of returning only those log lines that are relevant to your query.

Using the Console Encoder

The preceding example included a bunch of functionality in relatively little code. Let's instead assume you want to log something a bit more human-readable, yet that has structure. Zap includes a console encoder that's essentially a drop-in replacement for its JSON encoder. Listing 13-8 uses the console encoder to write structured log entries to standard output.

```
--snip--

func Example_zapConsole() {
    zl := zap.New(
        zapcore.NewCore(
         ❶ zapcore.NewConsoleEncoder(encoderCfg),
           zapcore.Lock(os.Stdout),
         ❷ zapcore.InfoLevel,
        ),
    )
    defer func() { _ = zl.Sync() }()

    console := ❸zl.Named("[console]")
    console.Info("this is logged by the logger")
 ❹ console.Debug("this is below the logger's threshold and won't log")
    console.Error("this is also logged by the logger")

    // Output:
 ❺ // info   [console]   this is logged by the logger
    // error  [console]   this is also logged by the logger
}
```

*Listing 13-8: Writing structured logs using console encoding (*zap_test.go*)*

The console encoder ❶ uses tabs to separate fields. It takes instruction from your encoder configuration to determine which fields to include and how to format each.

Notice you don't pass the `zap.AddCaller` and `zap.Fields` options to the logger in this example. As a result, the log entries won't have `caller` and `version` fields. Log entries will include the `caller` field only if the logger has the `zap.AddCaller` option and the encoder configuration defines its `CallerKey`, as in Listing 13-6.

You name the logger ❸ and write three log entries, each with a different log level. Since the logger's threshold is the `info` level ❷, the debug log entry ❹ does not appear in the output because `debug` is below the `info` threshold.

The output ❺ lacks key names but includes the field values delimited by a tab character. Although not obvious in print, there's a tab character between the log level, the log name, and the log message. If you type this into your editor, be mindful to add tab characters between those elements lest the example fail when you run it.

Logging with Different Outputs and Encodings

Zap includes useful functions that allow you to concurrently log to different outputs, using different encodings, at different log levels. Listing 13-9 creates a logger that writes JSON to a log file and console encoding to standard output. The logger writes only the debug log entries to the console.

```
--snip--

func Example_zapInfoFileDebugConsole() {
    logFile := ❶new(bytes.Buffer)
    zl := zap.New(
        zapcore.NewCore(
            zapcore.NewJSONEncoder(encoderCfg),
            zapcore.Lock(❷zapcore.AddSync(logFile)),
            zapcore.InfoLevel,
        ),
    )
    defer func() { _ = zl.Sync() }()

 ❸ zl.Debug("this is below the logger's threshold and won't log")
    zl.Error("this is logged by the logger")
```

*Listing 13-9: Using *bytes.Buffer as the log output and logging JSON to it (*zap_test.go*)*

You're using *bytes.Buffer ❶ to act as a mock log file. The only problem with this is that *bytes.Buffer does not have a Sync method and does not implement the zapcore.WriteSyncer interface. Thankfully, Zap includes a helper function named zapcore.AddSync ❷ that intelligently adds a no-op Sync method to an io.Writer. Aside from the use of this function, the rest of the logger implementation should be familiar to you. It's logging JSON to the log file and excluding any log entries below the info level. As a result, the first log entry ❸ should not appear in the log file at all.

Now that you have a logger writing JSON to a log file, let's experiment with Zap and create a new logger in Listing 13-10 that can simultaneously write JSON log entries to a log file and console log entries to standard output.

```
--snip--

    zl = ❶zl.WithOptions(
     ❷ zap.WrapCore(
            func(c zapcore.Core) zapcore.Core {
                ucEncoderCfg := encoderCfg
             ❸ ucEncoderCfg.EncodeLevel = zapcore.CapitalLevelEncoder
                return ❹zapcore.NewTee(
                    c,
                 ❺ zapcore.NewCore(
                        zapcore.NewConsoleEncoder(ucEncoderCfg),
                        zapcore.Lock(os.Stdout),
                        zapcore.DebugLevel,
                    ),
                )
            },
        ),
```

```
    )

    fmt.Println("standard output:")
 ❻ zl.Debug("this is only logged as console encoding")
    zl.Info("this is logged as console encoding and JSON")

    fmt.Print("\nlog file contents:\n", logFile.String())

    // Output:
    // standard output:
    // DEBUG   this is only logged as console encoding
    // INFO    this is logged as console encoding and JSON
    //
    // log file contents:
    // {"level":"error","msg":"this is logged by the logger"}
    // {"level":"info","msg":"this is logged as console encoding and JSON"}
}
```

*Listing 13-10: Extending the logger to log to multiple outputs (*zap_test.go*)*

Zap's `WithOptions` method ❶ clones the existing logger and configures the clone with the given options. You can use the `zap.WrapCore` function ❷ to modify the underlying `zap.Core` of the cloned logger. To mix things up, you make a copy of the encoder configuration and tweak it to instruct the encoder to output the level using all capital letters ❸. Lastly, you use the `zapcore.NewTee` function, which is like the `io.MultiWriter` function, to return a `zap.Core` that writes to multiple cores ❹. In this example, you're passing in the existing core and a new core ❺ that writes `debug`-level log entries to standard output.

When you use the cloned logger, both the log file and standard output receive any log entry at the `info` level or above, whereas only standard output receives debugging log entries ❻.

Sampling Log Entries

One of my warnings to you with regard to logging is to consider how it impacts your application from a CPU and I/O perspective. You don't want logging to become your application's bottleneck. This normally means taking special care when logging in the busy parts of your application.

One method to mitigate the logging overhead in critical code paths, such as a loop, is to sample log entries. It may not be necessary to log each entry, especially if your logger is outputting many duplicate log entries. Instead, try logging every *n*th occurrence of a duplicate entry.

Conveniently, Zap has a logger that does just that. Listing 13-11 creates a logger that will constrain its CPU and I/O overhead by logging a subset of log entries.

```
--snip--

func Example_zapSampling() {
    zl := zap.New(
     ❶ zapcore.NewSamplerWithOptions(
```

```
            zapcore.NewCore(
                zapcore.NewJSONEncoder(encoderCfg),
                zapcore.Lock(os.Stdout),
                zapcore.DebugLevel,
            ),
          ❷time.Second, ❸1, ❹3,
        ),
    )
    defer func() { _ = zl.Sync() }()

    for i := 0; i < 10; i++ {
        if i == 5 {
         ❺ time.Sleep(time.Second)
        }
     ❻ zl.Debug(fmt.Sprintf("%d", i))
     ❼ zl.Debug("debug message")
    }

    // ❽Output:
    // {"level":"debug","msg":"0"}
    // {"level":"debug","msg":"debug message"}
    // {"level":"debug","msg":"1"}
    // {"level":"debug","msg":"2"}
    // {"level":"debug","msg":"3"}
    // {"level":"debug","msg":"debug message"}
    // {"level":"debug","msg":"4"}
    // {"level":"debug","msg":"5"}
    // {"level":"debug","msg":"debug message"}
    // {"level":"debug","msg":"6"}
    // {"level":"debug","msg":"7"}
    // {"level":"debug","msg":"8"}
    // {"level":"debug","msg":"debug message"}
    // {"level":"debug","msg":"9"}
}
```

*Listing 13-11: Logging a subset of log entries to limit CPU and I/O overhead (*zap_test.go*)*

The `NewSamplerWithOptions` function ❶ wraps `zap.Core` with sampling functionality. It requires three additional arguments: a sampling interval ❷, the number of initial duplicate log entries to record ❸, and an integer ❹ representing the *n*th duplicate log entry to record after that point. In this example, you are logging the first log entry, and then every third duplicate log entry that the logger receives in a one-second interval. Once the interval elapses, the logger starts over and logs the first entry, then every third duplicate for the remainder of the one-second interval.

Let's look at this in action. You make 10 iterations around a loop. Each iteration logs both the counter ❻ and a generic debug message ❼, which stays the same for each iteration. On the sixth iteration, the example sleeps for one second ❺ to ensure that the sample logger starts logging anew during the next one-second interval.

Examining the output ❽, you see that the debug message prints during the first iteration and not again until the logger encounters the third duplicate debug message during the fourth loop iteration. But on the sixth

iteration, the example sleeps, and the sample logger ticks over to the next one-second interval, starting the logging over. It logs the first debug message of the interval in the sixth loop iteration and the third duplicate debug message in the ninth iteration of the loop.

Granted, this is a contrived example, but one that illustrates how to use this log-sampling technique as a compromise in CPU- and I/O-sensitive portions of your code. One place this technique may be applicable is when sending work to worker goroutines. Although you may send work as fast as the workers can handle it, you might want periodic updates on each worker's progress without having to incur too much logging overhead. The sample logger allows you to temper the log output and strike a balance between timely updates and minimal overhead.

Performing On-Demand Debug Logging

If debug logging introduces an unacceptable burden on your application under normal operation, or if the sheer amount of debug log data overwhelms your available storage space, you might want the ability to enable debug logging on demand. One technique is to use a semaphore file to enable debug logging. A *semaphore file* is an empty file whose existence is meant as a signal to the logger to change its behavior. If the semaphore file is present, the logger outputs `debug`-level logs. Once you remove the semaphore file, the logger reverts to its previous log level.

Let's use the `fsnotify` package to allow your application to watch for filesystem notifications. In addition to the standard library, the `fsnotify` package uses the `x/sys` package. Before you start writing code, let's make sure our `x/sys` package is current:

```
$ go get -u golang.org/x/sys/...
```

Not all logging packages provide safe methods to asynchronously modify log levels. Be aware that you may introduce a race condition if you attempt to modify a logger's level at the same time that the logger is reading the log level. The Zap logger allows you to retrieve a `sync/atomic`-based leveler to dynamically modify a logger's level while avoiding race conditions. You'll pass the atomic leveler to the `zapcore.NewCore` function in place of a log level, as you've previously done.

The `zap.AtomicLevel` struct implements the `http.Handler` interface. You can integrate it into an API and dynamically change the log level over HTTP instead of using a semaphore.

Listing 13-12 begins an example of dynamic logging using a semaphore file. You'll implement this example over the next few listings.

```
--snip--

func Example_zapDynamicDebugging() {
    tempDir, err := ioutil.TempDir("", "")
    if err != nil {
        log.Fatal(err)
    }
```

```
    defer func() { _ = os.RemoveAll(tempDir) }()

    debugLevelFile := ❶filepath.Join(tempDir, "level.debug")
    atomicLevel := ❷zap.NewAtomicLevel()

    zl := zap.New(
        zapcore.NewCore(
            zapcore.NewJSONEncoder(encoderCfg),
            zapcore.Lock(os.Stdout),
         ❸ atomicLevel,
        ),
    )
    defer func() { _ = zl.Sync() }()
```

*Listing 13-12: Creating a new logger using an atomic leveler (*zap_test.go*)*

Your code will watch for the *level.debug* file ❶ in the temporary directory. When the file is present, you'll dynamically change the logger's level to `debug`. To do that, you need a new atomic leveler ❷. By default, the atomic leveler uses the `info` level, which suits this example just fine. You pass in the atomic leveler ❸ when creating the core as opposed to specifying a log level itself.

Now that you have an atomic leveler and a place to store your semaphore file, let's write the code that will watch for semaphore file changes in Listing 13-13.

```
--snip--

    watcher, err := ❶fsnotify.NewWatcher()
    if err != nil {
        log.Fatal(err)
    }
    defer func() { _ = watcher.Close() }()

    err = ❷watcher.Add(tempDir)
    if err != nil {
        log.Fatal(err)
    }

    ready := make(chan struct{})
    go func() {
        defer close(ready)

        originalLevel := ❸atomicLevel.Level()

        for {
            select {
            case event, ok := ❹<-watcher.Events:
                if !ok {
                    return
                }
                if event.Name == ❺debugLevelFile {
                    switch {
                    case event.Op&fsnotify.Create == ❻fsnotify.Create:
                        atomicLevel.SetLevel(zapcore.DebugLevel)
```

```
                    ready <- struct{}{}
                case event.Op&fsnotify.Remove == ❼fsnotify.Remove:
                    atomicLevel.SetLevel(originalLevel)
                    ready <- struct{}{}
                }
            }
        case err, ok := ❽<-watcher.Errors:
            if !ok {
                return
            }
            zl.Error(err.Error())
        }
    }
}()
```

*Listing 13-13: Watching for any changes to the semaphore file (*zap_test.go*)*

First, you create a filesystem watcher ❶, which you'll use to watch the temporary directory ❷. The watcher will notify you of any changes to or within that directory. You also want to capture the current log level ❸ so that you can revert to it when you remove the semaphore file.

Next, you listen for events from the watcher ❹. Since you're watching a directory, you filter out any event unrelated to the semaphore file ❺ itself. Even then, you're interested in only the creation of the semaphore file or its removal. If the event indicates the creation of the semaphore file ❻, you change the atomic leveler's level to `debug`. If you receive a semaphore file removal event ❼, you set the atomic leveler's level back to its original level.

If you receive an error from the watcher ❽ at any point, you log it at the `error` level.

Let's see how this works in practice. Listing 13-14 tests the logger with and without the semaphore file present.

```
--snip--

❶ zl.Debug("this is below the logger's threshold")

    df, err := ❷os.Create(debugLevelFile)
    if err != nil {
        log.Fatal(err)
    }
    err = df.Close()
    if err != nil {
        log.Fatal(err)
    }
    <-ready

❸ zl.Debug("this is now at the logger's threshold")

    err = ❹os.Remove(debugLevelFile)
    if err != nil {
        log.Fatal(err)
    }
    <-ready
```

```
 ❺ zl.Debug("this is below the logger's threshold again")
 ❻ zl.Info("this is at the logger's current threshold")

    // Output:
    // {"level":"debug","msg":"this is now at the logger's threshold"}
    // {"level":"info","msg":"this is at the logger's current threshold"}
}
```

*Listing 13-14: Testing the logger's use of the semaphore file (*zap_test.go*)*

The logger's current log level via the atomic leveler is `info`. Therefore, the logger does not write the initial debug log entry ❶ to standard output. But if you create the semaphore file ❷, the code in Listing 13-13 should dynamically change the logger's level to `debug`. If you add another debug log entry ❸, the logger should write it to standard output. You then remove the semaphore file ❹ and write both a debug log entry ❺ and an info log entry ❻. Since the semaphore file no longer exists, the logger should write only the info log entry to standard output.

Scaling Up with Wide Event Logging

Wide event logging is a technique that creates a single, structured log entry per event to summarize the transaction, instead of logging numerous entries as the transaction progresses. This technique is most applicable to request-response loops, such as API calls, but it can be adapted to other use cases. When you summarize transactions in a structured log entry, you reduce the logging overhead while conserving the ability to index and search for transaction details.

One approach to wide event logging is to wrap an API handler in middleware. But first, since the `http.ResponseWriter` is a bit stingy with its output, you need to create your own response writer type (Listing 13-15) to collect and log the response code and length.

```
package ch13

import (
    "io"
    "io/ioutil"
    "net"
    "net/http"
    "net/http/httptest"
    "os"

    "go.uber.org/zap"
    "go.uber.org/zap/zapcore"
)

type wideResponseWriter struct {
 ❶ http.ResponseWriter
    length, status int
}
```

```
func (w *wideResponseWriter) ❷WriteHeader(status int) {
    w.ResponseWriter.WriteHeader(status)
    w.status = status
}

func (w *wideResponseWriter) ❸Write(b []byte) (int, error) {
    n, err := w.ResponseWriter.Write(b)
    w.length += n

    if w.status == 0 {
        w.status = ❹http.StatusOK
    }

    return n, err
}
```

*Listing 13-15: Creating a `ResponseWriter` to capture the response status code and length (*wide_test.go*)*

The new type embeds an object that implements the `http.ResponseWriter` interface ❶. In addition, you add `length` and `status` fields, since those values are ultimately what you want to log from the response. You override the `WriteHeader` method ❷ to easily capture the status code. Likewise, you override the `Write` method ❸ to keep an accurate accounting of the number of written bytes and optionally set the status code ❹ should the caller execute `Write` before `WriteHeader`.

Listing 13-16 uses your new type in wide event logging middleware.

```
--snip--

func WideEventLog(logger *zap.Logger, next http.Handler) http.Handler {
    return http.HandlerFunc(
        func(w http.ResponseWriter, r *http.Request) {
            wideWriter := ❶&wideResponseWriter{ResponseWriter: w}

          ❷ next.ServeHTTP(wideWriter, r)

            addr, _, _ := net.SplitHostPort(r.RemoteAddr)
          ❸ logger.Info("example wide event",
                zap.Int("status_code", wideWriter.status),
                zap.Int("response_length", wideWriter.length),
                zap.Int64("content_length", r.ContentLength),
                zap.String("method", r.Method),
                zap.String("proto", r.Proto),
                zap.String("remote_addr", addr),
                zap.String("uri", r.RequestURI),
                zap.String("user_agent", r.UserAgent()),
            )
        },
    )
}
```

*Listing 13-16: Implementing wide event logging middleware (*wide_test.go*)*

The wide event logging middleware accepts both a `*zap.Logger` and an `http.Handler` and returns an `http.Handler`. If this pattern is unfamiliar to you, please read "Handlers" on page 193.

First, you embed the `http.ResponseWriter` in a new instance of your wide event logging–aware response writer ❶. Then, you call the `ServeHTTP` method of the next `http.Handler` ❷, passing in your response writer. Finally, you make a single log entry ❸ with various bits of data about the request and response.

Keep in mind that I'm taking care here to omit values that would change with each execution and break the example output, like call duration. You would likely have to write code to deal with these in a real implementation.

Listing 13-17 puts the middleware into action and demonstrates the expected output.

```
--snip--

func Example_wideLogEntry() {
    zl := zap.New(
        zapcore.NewCore(
            zapcore.NewJSONEncoder(encoderCfg),
            zapcore.Lock(os.Stdout),
            zapcore.DebugLevel,
        ),
    )
    defer func() { _ = zl.Sync() }()

    ts := httptest.NewServer(
     ❶ WideEventLog(zl, http.HandlerFunc(
            func(w http.ResponseWriter, r *http.Request) {
                defer func(r io.ReadCloser) {
                    _, _ = io.Copy(ioutil.Discard, r)
                    _ = r.Close()
                }(r.Body)
                _, _ = ❷w.Write([]byte("Hello!"))
            },
        )),
    )
    defer ts.Close()

    resp, err := ❸http.Get(ts.URL + "/test")
    if err != nil {
     ❹ zl.Fatal(err.Error())
    }
    _ = resp.Body.Close()

    // ❺Output:
    // {"level":"info","msg":"example wide event","status_code":200,
"response_length":6,"content_length":0,"method":"GET","proto":"HTTP/1.1",
"remote_addr":"127.0.0.1","uri":"/test","user_agent":"Go-http-client/1.1"}
}
```

*Listing 13-17: Using the wide event logging middleware to log the details of a GET call (*wide_test.go*)*

As in Chapter 9, you use the `httptest` server with your `WideEventLog` middleware ❶. You pass `*zap.Logger` into the middleware as the first argument and `http.Handler` as the second argument. The handler writes a simple *Hello!* to the response ❷ so the response length is nonzero. That way, you can prove that your response writer works. The logger writes the log entry immediately before you receive the response to your GET request ❸. As before, I must wrap the JSON output ❺ for printing in this book, but it consumes a single line otherwise.

Since this is just an example, I elected to use the logger's `Fatal` method ❹, which writes the error message to the log file and calls `os.Exit(1)` to terminate the application. You shouldn't use this in code that is supposed to keep running in the event of an error.

Log Rotation with Lumberjack

If you elect to output log entries to a file, you could leverage an application like *logrotate* to keep them from consuming all available hard drive space. The downside to using a third-party application to manage log files is that the third-party application will need to signal to your application to reopen its log file handle lest your application keep writing to the rotated log file.

A less invasive and more reliable option is to add log file management directly to your logger by using a library like *Lumberjack*. Lumberjack handles log rotation in a way that is transparent to the logger, because your logger treats Lumberjack as any other `io.Writer`. Meanwhile, Lumberjack keeps track of the log entry accounting and file rotation for you.

Listing 13-18 adds log rotation to a typical Zap logger implementation.

```
--snip--

func TestZapLogRotation(t *testing.T) {
    tempDir, err := ioutil.TempDir("", "")
    if err != nil {
        t.Fatal(err)
    }
    defer func() { _ = os.RemoveAll(tempDir) }()

    zl := zap.New(
        zapcore.NewCore(
            zapcore.NewJSONEncoder(encoderCfg),
         ❶ zapcore.AddSync(
             ❷ &lumberjack.Logger{
                    Filename:   ❸filepath.Join(tempDir, "debug.log"),
                    Compress:   ❹true,
                    LocalTime:  ❺true,
                    MaxAge:     ❻7,
                    MaxBackups: ❼5,
                    MaxSize:    ❽100,
                },
            ),
            zapcore.DebugLevel,
        ),
    )
```

```
    defer func() { _ = zl.Sync() }()

    zl.Debug("debug message written to the log file")
}
```

*Listing 13-18: Adding log rotation to the Zap logger using Lumberjack (*zap_test.go*)*

Like the `*bytes.Buffer` in Listing 13-9, `*lumberjack.Logger` ❷ does not implement the `zapcore.WriteSyncer`. It, too, lacks a `Sync` method. Therefore, you need to wrap it in a call to `zapcore.AddSync` ❶.

Lumberjack includes several fields to configure its behavior, though its defaults are sensible. It uses a log filename in the format *<processname>-lumberjack.log*, saved in the temporary directory, unless you explicitly give it a log filename ❸. You can also elect to save hard drive space and have Lumberjack compress ❹ rotated log files. Each rotated log file is time-stamped using UTC by default, but you can instruct Lumberjack to use local time ❺ instead. Finally, you can configure the maximum log file age before it should be rotated ❻, the maximum number of rotated log files to keep ❼, and the maximum size in megabytes ❽ of a log file before it should be rotated.

You can continue using the logger as if it were writing directly to standard output or `*os.File`. The difference is that Lumberjack will intelligently handle the log file management for you.

Instrumenting Your Code

Instrumenting your code is the process of collecting metrics for the purpose of making inferences about the current state of your service—such as the duration of each request-response loop, the size of each response, the number of connected clients, the latency between your service and a third-party API, and so on. Whereas logs provide a record of how your service got into a certain state, metrics give you insight into that state itself.

Instrumentation is easy, so much so that I'm going to give you the opposite advice I did for logging: instrument everything (initially). Fine-grained instrumentation involves hardly any overhead, it's efficient to ship, and it's inexpensive to store. Plus, instrumentation can solve one of the challenges of logging I mentioned earlier: that you won't initially know all the questions you'll want to ask, particularly for complex systems. An insidious problem may be ready to ruin your weekend because you lack critical metrics to give you an early warning that something is wrong.

This section will introduce you to metric types and show you the basics for using those types in your services. You will learn about Go kit's `metrics` package, which is an abstraction layer that provides useful interfaces for popular metrics platforms. You'll round out the instrumentation by using Prometheus as your target metrics platform and set up an endpoint for Prometheus to scrape. If you elect to use a different platform

in the future, you will need to swap out only the Prometheus bits of this code; you could leave the Go kit code as is. If you're just getting started with instrumentation, one option is to use Grafana Cloud at *https://grafana.com/products/cloud/* to scrape and visualize your metrics. Its free tier is adequate for experimenting with instrumentation.

Setup

To abstract the implementation of your metrics and the packages they depend on, let's begin by putting them in their own package (Listing 13-19).

```
package metrics

import (
    "flag"

 ❶ "github.com/go-kit/kit/metrics"
 ❷ "github.com/go-kit/kit/metrics/prometheus"
 ❸ prom "github.com/prometheus/client_golang/prometheus"
)

var (
    Namespace = ❹flag.String("namespace", "web", "metrics namespace")
    Subsystem = ❺flag.String("subsystem", "server1", "metrics subsystem")
```

*Listing 13-19: Imports and command line flags for the metrics example (*instrumentation/metrics/metrics.go*)*

You import Go kit's `metrics` package ❶, which provides the interfaces your code will use, its `prometheus` adapter ❷ so you can use Prometheus as your metrics platform, and Go's Prometheus client package ❸ itself. All Prometheus-related imports reside in this package. The rest of your code will use Go kit's interfaces. This allows you to swap out the underlying metrics platform without the need to change your code's instrumentation.

Prometheus prefixes its metrics with a namespace and a subsystem. You could use the service name for the namespace and the node or hostname for the subsystem, for example. In this example, you'll use `web` for the namespace ❹ and `server1` for the subsystem ❺ by default. As a result, your metrics will use the `web_server1_` prefix. You'll see this prefix in Listing 13-30's command line output.

Now let's explore the various metric types, starting with counters.

Counters

Counters are used for tracking values that only increase, such as request counts, error counts, or completed task counts. You can use a counter to calculate the rate of increase for a given interval, such as the number of connections per minute.

Listing 13-20 defines two counters: one to track the number of requests and another to account for the number of write errors.

```
--snip--

    Requests ❶metrics.Counter = ❷prometheus.NewCounterFrom(
     ❸ prom.CounterOpts{
            Namespace: *Namespace,
            Subsystem: *Subsystem,
            Name:     ❹"request_count",
            Help:     ❺"Total requests",
        },
        []string{},
    )

    WriteErrors metrics.Counter = prometheus.NewCounterFrom(
        prom.CounterOpts{
            Namespace: *Namespace,
            Subsystem: *Subsystem,
            Name:      "write_errors_count",
            Help:      "Total write errors",
        },
        []string{},
    )
```

*Listing 13-20: Creating counters as Go kit interfaces (*instrumentation/metrics/metrics.go*)*

Each counter implements Go kit's `metrics.Counter` interface ❶. The concrete type for each counter comes from Go kit's `prometheus` adapter ❷ and relies on a `CounterOpts` struct ❸ from the Prometheus client package for configuration. Aside from the namespace and subsystem values we covered, the other important values you set are the metric name ❹ and its help string ❺, which describes the metric.

Gauges

Gauges allow you to track values that increase or decrease, such as the current memory usage, in-flight requests, queue sizes, fan speed, or the number of ThinkPads on my desk. Gauges do not support rate calculations, such as the number of connections per minute or megabits transferred per second, while counters do.

Listing 13-21 creates a gauge to track open connections.

```
--snip--

    OpenConnections ❶metrics.Gauge = ❷prometheus.NewGaugeFrom(
     ❸ prom.GaugeOpts{
            Namespace: *Namespace,
            Subsystem: *Subsystem,
            Name:      "open_connections",
            Help:      "Current open connections",
        },
        []string{},
    )
```

*Listing 13-21: Creating a gauge as a Go kit interface (*instrumentation/metrics/metrics.go*)*

Creating a gauge is much like creating a counter. You create a new variable of Go kit's `metrics.Gauge` interface ❶ and use the `NewGaugeFrom` function ❷ from Go kit's `prometheus` adapter to create the underlying type. The Prometheus client's `GaugeOpts` struct ❸ provides the settings for your new gauge.

Histograms and Summaries

A *histogram* places values into predefined buckets. Each bucket is associated with a range of values and named after its maximum one. When a value is observed, the histogram increments the maximum value of the smallest bucket into which the value fits. In this way, a histogram tracks the frequency of observed values for each bucket.

Let's look at a quick example. Assuming you have three buckets valued at 0.5, 1.0, and 1.5, if a histogram observes the value 0.42, it will increment the counter associated with bucket 0.5, because 0.5 is the smallest bucket that can contain 0.42. It covers the range of 0.5 and smaller values. If the histogram observes the value 1.23, it will increment the counter associated with the bucket 1.5, which covers values in the range of above 1.0 through 1.5. Naturally, the 1.0 bucket covers the range of above 0.5 through 1.0.

You can use a histogram's distribution of observed values to estimate a percentage or an average of all values. For example, you could use a histogram to calculate the average request sizes or response sizes observed by your service.

A *summary* is a histogram with a few differences. First, a histogram requires predefined buckets, whereas a summary calculates its own buckets. Second, the metrics server calculates averages or percentages from histograms, whereas your service calculates the averages or percentages from summaries. As a result, you can aggregate histograms across services on the metrics server, but you cannot do the same for summaries.

The general advice is to use summaries when you don't know the range of expected values, but I'd advise you to use histograms whenever possible so that you can aggregate histograms on the metrics server. Let's use a histogram to observe request duration (see Listing 13-22).

```
--snip--

    RequestDuration ❶metrics.Histogram = ❷prometheus.NewHistogramFrom(
     ❸ prom.HistogramOpts{
            Namespace: *Namespace,
            Subsystem: *Subsystem,
            Buckets: ❹[]float64{
                0.0000001, 0.0000002, 0.0000003, 0.0000004, 0.0000005,
                0.000001, 0.0000025, 0.000005, 0.0000075, 0.00001,
                0.0001, 0.001, 0.01,
            },
            Name: "request_duration_histogram_seconds",
            Help: "Total duration of all requests",
        },
        []string{},
    )
)
```

*Listing 13-22: Creating a histogram metric (*instrumentation/metrics/metrics.go*)*

Both the summary and histogram metric types implement Go kit's `metrics.Histogram` interface ❶ from its `prometheus` adapter. Here, you're using a histogram metric type ❷, using the Prometheus client's `HistogramOpts` struct ❸ for configuration. Since Prometheus's default bucket sizes are too large for the expected request duration range when communicating over localhost, you define custom bucket sizes ❹. I encourage you to experiment with the number of buckets and bucket sizes.

If you'd rather implement `RequestDuration` as a summary metric, you can substitute the code in Listing 13-22 for the code in Listing 13-23.

```
--snip--

    RequestDuration ❶metrics.Histogram = prometheus.NewSummaryFrom(
        prom.SummaryOpts{
            Namespace: *Namespace,
            Subsystem: *Subsystem,
            Name: "request_duration_summary_seconds",
            Help: "Total duration of all requests",
        },
        []string{},
    )
)
```

Listing 13-23: Optionally creating a summary metric

As you can see, this looks a lot like a histogram, minus the `Bucket` method. Notice that you still use the `metrics.Histogram` interface ❶ with a Prometheus summary metric. This is because Go kit does not distinguish between histograms and summaries; only your implementation of the interface does.

Instrumenting a Basic HTTP Server

Let's combine these metric types in a practical example: instrumenting a Go HTTP server. The biggest challenges here are determining what you want to instrument, where best to instrument it, and what metric type is most appropriate for each value you want to track. If you use Prometheus for your metrics platform, as you'll do here, you'll also need to add an HTTP endpoint for the Prometheus server to scrape.

Listing 13-24 details the initial code needed for an application that comprises an HTTP server to serve the metrics endpoint and another HTTP server to pass all requests to an instrumented handler.

```
package main

import (
    "bytes"
    "flag"
    "fmt"
```

```
    "io"
    "io/ioutil"
    "log"
    "math/rand"
    "net"
    "net/http"
    "sync"
    "time"

 ❶ "github.com/prometheus/client_golang/prometheus/promhttp"

 ❷ "github.com/awoodbeck/gnp/ch13/instrumentation/metrics"
)

var (
    metricsAddr = ❸flag.String("metrics", "127.0.0.1:8081",
        "metrics listen address")
    webAddr = ❹flag.String("web", "127.0.0.1:8082", "web listen address")
)
```

*Listing 13-24: Imports and command line flags for the metrics example (*instrumentation/main.go*)*

The only imports your code needs are the `promhttp` package for the metrics endpoint and your `metrics` package to instrument your code. The `promhttp` package ❶ includes an `http.Handler` that a Prometheus server can use to scrape metrics from your application. This handler serves not only your metrics but also metrics related to the runtime, such as the Go version, number of cores, and so on. At a minimum, you can use the metrics provided by the Prometheus handler to gain insight into your service's memory utilization, open file descriptors, heap and stack details, and more.

All variables exported by your `metrics` package ❷ are Go kit interfaces. Your code doesn't need to concern itself with the underlying metrics platform or its implementation, only how these metrics are made available to the metrics server. In a real-world application, you could further abstract the Prometheus handler to fully remove any dependency other than your metrics package from the rest of your code. But in the interest of keeping this example succinct, I've included the Prometheus handler in the `main` package.

Now, onto the code you want to instrument. Listing 13-25 adds the function your web server will use to handle all incoming requests.

```
--snip--

func helloHandler(w http.ResponseWriter, _ *http.Request) {
 ❶ metrics.Requests.Add(1)
    defer func(start time.Time) {
     ❷ metrics.RequestDuration.Observe(time.Since(start).Seconds())
    }(time.Now())
```

```
    _, err := w.Write([]byte("Hello!"))
    if err != nil {
      ❸ metrics.WriteErrors.Add(1)
    }
}
```

Listing 13-25: An instrumented handler that responds with random latency (instrumentation/main.go)

Even in such a simple handler, you're able to make three meaningful measurements. You increment the requests counter upon entering the handler ❶ since it's the most logical place to account for it. You also immediately defer a function that calculates the request duration and uses the request duration summary metric to observe it ❷. Lastly, you account for any errors writing the response ❸.

Now, you need to put the handler to use. But first, you need a helper function that will allow you to spin up a couple of HTTP servers: one to serve the metrics endpoint and one to serve this handler. Listing 13-26 details such a function.

```
--snip--

func newHTTPServer(addr string, mux http.Handler,
    stateFunc ❶func(net.Conn, http.ConnState)) error {
    l, err := net.Listen("tcp", addr)
    if err != nil {
        return err
    }

    srv := &http.Server{
        Addr:              addr,
        Handler:           mux,
        IdleTimeout:       time.Minute,
        ReadHeaderTimeout: 30 * time.Second,
        ConnState:         stateFunc,
    }

    go func() { log.Fatal(srv.Serve(l)) }()

    return nil
}

func ❷connStateMetrics(_ net.Conn, state http.ConnState) {
    switch state {
    case http.StateNew:
      ❸ metrics.OpenConnections.Add(1)
    case http.StateClosed:
      ❹ metrics.OpenConnections.Add(-1)
    }
}
```

Listing 13-26: Functions to create an HTTP server and instrument connection states (instrumentation/main.go)

This HTTP server code resembles that of Chapter 9. The exception here is you're defining the server's `ConnState` field, accepting it as an argument ❶ to the `newHTTPServer` function.

The HTTP server calls its `ConnState` field anytime a network connection changes. You can leverage this functionality to instrument the number of open connections the server has at any one time. You can pass the `connStateMetrics` function ❷ to the `newHTTPServer` function anytime you want to initialize a new HTTP server and track its open connections. If the server establishes a new connection, you increment the open connections gauge ❸ by 1. If a connection closes, you decrement the gauge ❹ by 1. Go kit's gauge interface provides an `Add` method, so decrementing a value involves adding a negative number.

Let's create an example that puts all these pieces together. Listing 13-27 creates an HTTP server to serve up the Prometheus endpoint and another HTTP server to serve your instrumented handler.

```
--snip--

func main() {
    flag.Parse()
    rand.Seed(time.Now().UnixNano())

    mux := http.NewServeMux()
 ❶ mux.Handle("/metrics/", promhttp.Handler())
    if err := newHTTPServer(*metricsAddr, mux, ❷nil); err != nil {
        log.Fatal(err)
    }
    fmt.Printf("Metrics listening on %q ...\n", *metricsAddr)

    if err := newHTTPServer(*webAddr, ❸http.HandlerFunc(helloHandler),
      ❹connStateMetrics); err != nil {
        log.Fatal(err)
    }
    fmt.Printf("Web listening on %q ...\n\n", *webAddr)
```

*Listing 13-27: Starting two HTTP servers to serve `metrics` and the `helloHandler` (*instrumentation/main.go*)*

First, you spawn an HTTP server with the sole purpose of serving the Prometheus handler ❶ at the `/metrics/` endpoint where Prometheus scrapes metrics from by default. Since you do not pass in a function for the third argument ❷, this HTTP server won't have a function assigned to its `ConnState` field to call on each connection state change. Then, you spin up another HTTP server to handle each request with the `helloHandler` ❸. But this time, you pass in the `connStateMetrics` function ❹. As a result, this HTTP server will gauge open connections.

Now, you can spin up many HTTP clients to make a bunch of requests to affect your metrics (see Listing 13-28).

```
--snip--

    clients := ❶500
    gets := ❷100
    wg := new(sync.WaitGroup)

    fmt.Printf("Spawning %d connections to make %d requests each ...",
        clients, gets)
    for i := 0; i < clients; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()

            c := &http.Client{
                Transport: ❸http.DefaultTransport.(*http.Transport).Clone(),
            }

            for j := 0; j < gets; j++ {
                resp, err := ❹c.Get(fmt.Sprintf("http://%s/", *webAddr))
                if err != nil {
                    log.Fatal(err)
                }
                _, _ = ❺io.Copy(ioutil.Discard, resp.Body)
                _ = ❻resp.Body.Close()
            }
        }()
    }
❼ wg.Wait()
    fmt.Print(" done.\n\n")
```

*Listing 13-28: Instructing 500 HTTP clients to each make 100 GET calls (*instrumentation/main.go*)*

You start by spawning 500 HTTP clients ❶ to each make 100 GET calls ❷. But first, you need to address a problem. The `http.Client` uses the `http.DefaultTransport` if its `Transport` method is nil. The `http.DefaultTransport` does an outstanding job of caching TCP connections. If all 500 HTTP clients use the same transport, they'll all make calls over about two TCP sockets. Our open connections gauge would reflect the two idle connections when you're done with this example, which isn't really the goal.

Instead, you must make sure to give each HTTP client its own transport. Cloning the default transport ❸ is good enough for our purposes.

Now that each client has its own transport and you're assured each client will make its own TCP connection, you iterate through a GET call ❹ 100 times with each client. You must also be diligent about draining ❺ and closing ❻ the response body so each client can reuse its TCP connection.

Once all 500 HTTP clients complete their 100 calls ❼, you can move on to Listing 13-29 and check the current state of the metrics.

```
--snip--

    resp, err := ❶http.Get(fmt.Sprintf("http://%s/metrics", *metricsAddr))
    if err != nil {
        log.Fatal(err)
    }

    b, err := ioutil.ReadAll(resp.Body)
    if err != nil {
        log.Fatal(err)
    }
    _ = resp.Body.Close()

    metricsPrefix := ❷fmt.Sprintf("%s_%s", *metrics.Namespace,
        *metrics.Subsystem)
    fmt.Println("Current Metrics:")
    for _, line := range bytes.Split(b, []byte("\n")) {
        if ❸bytes.HasPrefix(line, []byte(metricsPrefix)) {
            fmt.Printf("%s\n", line)
        }
    }
}
```

*Listing 13-29: Displaying the current metrics matching your namespace and subsystem (*instrumentation/main.go*)*

You retrieve all the metrics from the metrics endpoint ❶. This will cause the metrics web server to return all metrics stored by the Prometheus client, in addition to details about each metric it tracks, which includes the metrics you added. Since you're interested in only your metrics, you can check each line starting with your namespace, an underscore, and your subsystem ❷. If the line matches this prefix ❸, you print it to standard output. Otherwise, you ignore the line and move on.

Let's run this example on the command line and examine the resulting metrics in Listing 13-30.

```
$ go run instrumentation/main.go
Metrics listening on "127.0.0.1:8081" ...
Web listening on "127.0.0.1:8082" ...

Spawning 500 connections to make 100 requests each ... done.

Current Metrics:
web_server1_open_connections ❶500
web_server1_request_count ❷50000
web_server1_request_duration_histogram_seconds_bucket{le="1e-07"} ❸0
web_server1_request_duration_histogram_seconds_bucket{le="2e-07"} 1
web_server1_request_duration_histogram_seconds_bucket{le="3e-07"} 613
web_server1_request_duration_histogram_seconds_bucket{le="4e-07"} 13591
web_server1_request_duration_histogram_seconds_bucket{le="5e-07"} 33216
web_server1_request_duration_histogram_seconds_bucket{le="1e-06"} 40183
```

```
web_server1_request_duration_histogram_seconds_bucket{le="2.5e-06"} 49876
web_server1_request_duration_histogram_seconds_bucket{le="5e-06"} 49963
web_server1_request_duration_histogram_seconds_bucket{le="7.5e-06"} 49973
web_server1_request_duration_histogram_seconds_bucket{le="1e-05"} 49979
web_server1_request_duration_histogram_seconds_bucket{le="0.0001"} 49994
web_server1_request_duration_histogram_seconds_bucket{le="0.001"} 49997
web_server1_request_duration_histogram_seconds_bucket{le="0.01"} ❹50000
web_server1_request_duration_histogram_seconds_bucket{le="+Inf"} 50000
web_server1_request_duration_histogram_seconds_sum ❺0.04102166899999979
web_server1_request_duration_histogram_seconds_count ❻50000
```

Listing 13-30: Web server output and resulting metrics

As expected, 500 connections were open ❶ at the time you queried the metrics. These connections are idle. You can experiment with the HTTP client by invoking its `CloseIdleConnections` method after it's done making 100 GET calls; see how that change affects the open connections gauge. Likewise, see what happens to the open connections when you don't define their `Transport` field.

The request count is 50,000 ❷, so all requests succeeded.

Do you notice what's missing? The write errors counter. Since no write errors occur, the write errors counter never increments. As a result, it doesn't show up in the metrics output. You could make a call to `metrics.WriteErrors.Add(0)` to make the metric show up without changing its value, but its absence probably bothers you more than it bothers Prometheus. Just be aware that the metrics output may not include all instrumented metrics, just the ones that have changed since initialization.

The underlying Prometheus histogram is a *cumulative* histogram: any value that increments a bucket's counter also increments the counters for all buckets less than the value. Therefore, you see increasing values in each bucket until you reach the 0.01 bucket ❹. Even though you define a range of buckets, Prometheus adds an infinite bucket for you. In this example, you defined a bucket smaller than all observed values ❸, so its counter is still zero.

A histogram and a summary maintain two additional counters: the sum of all observed values ❺ and the total number of observed values ❻. If you use a summary, the Prometheus endpoint will present only these two counters. It will not detail the summary's buckets as it does with a histogram. Therefore, the Prometheus server can aggregate histogram buckets but cannot do the same for summaries.

What You've Learned

Logging is hard. Instrumentation, not so much. Be frugal with your logging and generous with your instrumentation. Logging isn't free and can quickly add latency if you aren't mindful of where and how much you log. You cannot go wrong by logging actionable items, particularly ones that should trigger an alert. On the other hand, instrumentation is very efficient. You should instrument everything, at least initially. Metrics detail the current state of your

service and provide insight into potential problems, whereas logs provide an immutable audit trail of sorts that explains the current state of your service and helps you diagnose failures.

Go's `log` package provides enough functionality to satisfy basic log requirements. But it becomes cumbersome when you need to log to more than one output or at varying levels of verbosity. At that point, you're better off with a comprehensive solution such as Uber's Zap logger. No matter what logger you use, consider adding structure to your log entries by including additional metadata. Structured logging allows you to leverage software to quickly filter and search log entries, particularly if you centralize logs across your infrastructure.

On-demand debug logging and wide event logging are two methods you can use to collect important information while minimizing logging's impact on the performance of your service. You can use the creation of a semaphore file to signal your logger to enable debug logging. When you remove the semaphore file, the logger immediately disables debug logging. Wide event logs summarize events in a request-response loop. You can replace numerous log entries with a single wide event log without hindering your ability to diagnose failures.

One approach to instrumentation is to use Go kit's `metrics` package, which provides interfaces for common metric types and adapters for popular metrics platforms. It allows you to abstract the details of each metrics platform away from your instrumented code.

The `metrics` package supports counters, gauges, histograms, and summaries. Counters monotonically increase and can be used to calculate rates of change. Use counters to track values like request counts, error counts, or completed tasks. Gauges track values that can increase and decrease, such as current memory usage, in-flight requests, and queue sizes. Histograms and summaries place observed values in buckets and allow you to estimate averages or percentages of all values. You could use a histogram or summary to approximate the average request duration or response size.

Taken together, logging and metrics give you necessary insight into your service, allowing you to proactively address potential problems and recover from failures.

14

MOVING TO THE CLOUD

In August of 2006, Amazon Web Services (AWS) brought public cloud infrastructure to the mainstream when it introduced its virtual computer, Elastic Compute Cloud (EC2). EC2 removed barriers to providing services over the internet; you no longer needed to purchase servers and software licenses, sign support contracts, rent office space, or hire IT professionals to maintain your infrastructure. Instead, you paid AWS as needed for the use of EC2 instances, allowing you to scale your business while AWS handled the maintenance, redundancy, and standards compliance details for you. In the following years, both Google and Microsoft released public cloud offerings to compete with AWS. Now all three cloud providers offer comprehensive services that cover everything from analytics to storage.

The goal of this chapter is to give you an apples-to-apples comparison of Amazon Web Services, Google Cloud, and Microsoft Azure. You'll create and deploy an application to illustrate the differences in each provider's tooling, authentication, and deployment experience. Your application will follow the *platform-as-a-service (PaaS)* model, in which you create the application and deploy it on the cloud provider's platform. Specifically, you'll create a function and deploy it to AWS Lambda, Google Cloud Functions, and Microsoft Azure Functions. We'll stick to the command line as much as possible to keep the comparisons relative and introduce you to each provider's tooling.

All three service providers offer a trial period, so you shouldn't incur any costs. If you've exhausted your trial, please keep potential costs in mind as you work through the following sections.

You'll create a simple function that retrieves the URL of the latest XKCD comic, or optionally the previous comic. This will demonstrate how to retrieve data from within the function to fulfill the client's request and persist function state between executions.

By the end of this chapter, you should feel comfortable writing an application, deploying it, and testing it to leverage the PaaS offerings of AWS, Google Cloud, and Microsoft Azure. You should have a better idea of which provider's workflow best fits your use case if you choose to make the jump to the cloud.

Laying Some Groundwork

The XKCD website offers a Real Simple Syndication (RSS) feed at *https://xkcd.com/rss.xml*. As its file extension indicates, the feed uses XML. You can use Go's *encoding/xml* package to parse the feed.

Before you deploy a function to the cloud that can retrieve the URL of the latest XKCD comic, you need to write some code that will allow you to make sense of the RSS feed. Listing 14-1 creates two types for parsing the feed.

```
package feed

import (
    "context"
    "encoding/xml"
    "fmt"
    "io/ioutil"
    "net/http"
)

type Item struct {
    Title     string `xml:"title"`
    URL       string `xml:"link"`
    Published string ❶`xml:"pubDate"`
}
```

```
type RSS struct {
    Channel struct {
        Items []Item `xml:"item"`
    } `xml:"channel"`
    entityTag ❷string
}
```

*Listing 14-1: Structure that represents the XKCD RSS feed (*feed/rss.go*)*

The `RSS` struct represents the RSS feed, and the `Item` struct represents each item (comic) in the feed. Like Go's *encoding/json* package you used in earlier chapters, its *encoding/xml* package can use struct tags to map XML tags to their corresponding struct fields. For example, the `Published` field's tag ❶ instructs the *encoding/xml* package to assign it the item's `pubDate` value.

It's important to be a good internet neighbor and keep track of the feed's entity tag ❷. Web servers often derive entity tags for content that may not change from one request to another. Clients can track these entity tags and present them with future requests. If the server determines that the requested content has the same entity tag, it can forgo returning the entire payload and return a 304 Not Modified status code so the client knows to use its cached copy instead. You'll use this value in Listing 14-2 to conditionally update the `RSS` struct when the feed changes.

```
--snip--
func (r RSS) Items() []Item {
    items := ❶make([]Item, len(r.Channel.Items))
    copy(items, r.Channel.Items)

    return items
}

func (r *RSS) ParseURL(ctx context.Context, u string) error {
    req, err := http.NewRequestWithContext(ctx, http.MethodGet, u, nil)
    if err != nil {
        return err
    }

    if r.entityTag != "" {
     ❷ req.Header.Add("ETag", r.entityTag)
    }

    resp, err := http.DefaultClient.Do(req)
    if err != nil {
        return err
    }

    switch resp.StatusCode {
    case ❷http.StatusNotModified: // no-op
    case ❸http.StatusOK:
        b, err := ioutil.ReadAll(resp.Body)
        if err != nil {
```

```
            return err
        }
        _ = resp.Body.Close()

        err = xml.Unmarshal(b, r)
        if err != nil {
            return err
        }

        r.entityTag = ❹resp.Header.Get("ETag")
    default:
        return fmt.Errorf("unexpected status code: %v", resp.StatusCode)
    }

    return nil
}
```

*Listing 14-2: Methods to parse the XKCD RSS feed and return a slice of items (*feed/rss.go*)*

There are three things to note here. First, the `RSS` struct and its methods are not safe for concurrent use. This isn't a concern for your use case, but it's best that you're aware of this fact. Second, the `Items` method returns a slice of the items in the `RSS` struct, which is empty until your code calls the `ParseURL` method to populate the `RSS` struct. Third, the `Items` method makes a copy of the `Items` slice ❶ and returns the copy to prevent possible corruption of the original `Items` slice. This is also a bit of overkill for your use case, but it's best to be aware that you're returning a reference type that the receiver can modify. If the receiver modifies the copy, it won't affect your original.

Parsing the RSS feed is straightforward and should look familiar. The `ParseURL` method retrieves the RSS feed by using a `GET` call. If the feed is new, the method reads the XML from the response body and invokes the `xml.Unmarshal` function to populate the `RSS` struct with the XML in the server.

Notice you conditionally set the request's `ETag` header ❷ so the XKCD server can determine whether it needs to send the feed contents or you currently have the latest version. If the server responds with a 304 Not Modified status code, the `RSS` struct remains unchanged. If you receive a 200 OK ❸, you received a new version of the feed and unmarshal the response body's XML into the `RSS` struct. If successful, you update the entity tag ❹.

With this logic in place, the `RSS` struct should update itself only if its entity tag is empty, as it would be on initialization of the struct, or if a new feed is available.

The last task is to create a *go.mod* file by using the following commands:

```
$ cd feed
feed$ go mod init github.com/awoodbeck/gnp/ch14/feed
go: creating new go.mod: module github.com/awoodbeck/gnp/ch14/feed
feed$ cd -
```

These commands initialize a new module named *github.com/awoodbeck/gnp/ch14/feed*, which will be used by code later in this chapter.

AWS Lambda

AWS Lambda is a serverless platform that offers first-class support for Go. You can create Go applications, deploy them, and let Lambda handle the implementation details. It will scale your code to meet demand. Before you can get started with Lambda, please make sure you create a trial account at *https://aws.amazon.com/*.

Installing the AWS Command Line Interface

AWS offers version 2 of its command line interface (CLI) tools for Windows, macOS, and Linux. You can find detailed instructions for installing them at *https://docs.aws.amazon.com/cli/latest/userguide/cli-chap-install.html*.

Use the following commands to install the AWS CLI tools on Linux:

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" \
-o "awscliv2.zip"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time
Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 32.3M  100 32.3M    0     0  31.1M      0  0:00:01  0:00:01 --:--:-- 31.1M
$ unzip -q awscliv2.zip
$ sudo ./aws/install
[sudo] password for user:
You can now run: /usr/local/bin/aws --version
$ aws --version
aws-cli/2.0.56 Python/3.7.3 Linux/5.4.0-7642-generic exe/x86_64.pop.20
```

Download the AWS CLI version 2 archive. Use `curl` to download the ZIP file from the command line. Then unzip the archive and use `sudo` to run the *./aws/install* executable. Once it's complete, run `aws --version` to verify that the AWS binary is in your path and that you're running version 2.

Configuring the CLI

Now that you have the AWS CLI installed, you need to configure it with credentials so it can interact with AWS on your account's behalf. This section walks you through that process. If you get confused, review the AWS CLI configuration quick-start guide at *https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-quickstart.html*.

First, access the AWS Console at *https://console.aws.amazon.com*. Log into the AWS Console to access the drop-down menu shown in Figure 14-1.

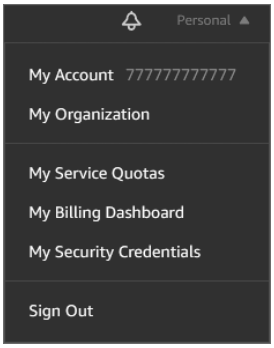


Figure 14-1: Accessing your AWS account security credentials

Click your account name in the upper-right corner of the AWS Console (**Personal** in Figure 14-1). Then, select **My Security Credentials** from the drop-down menu. This link should take you to the Your Security Credentials page, shown in Figure 14-2.

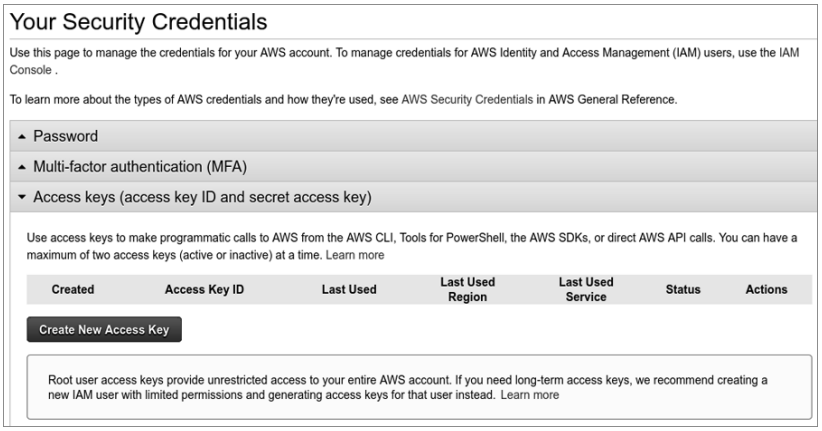


Figure 14-2: Creating a new access key

Select the **Access keys** section heading to expand the section. Then click the **Create New Access Key** button to create credentials to use on the command line. This will display the credentials (Figure 14-3).

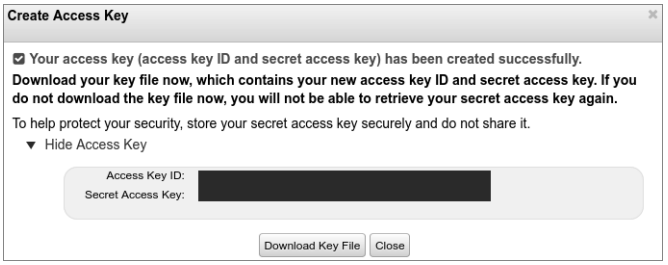


Figure 14-3: Retrieving the new access Key ID and secret access key

You'll need both the access Key ID and secret access key values to authenticate your commands on the command line with AWS. Make sure you download the key file and keep it in a secure place in case you need to retrieve it in the future. For now, you'll use them to configure your AWS command line interface:

```
$ aws configure
AWS Access Key ID [None]: AIDA1111111111EXAMPLE
AWS Secret Access Key [None]: YxMCBWETtZjZhW6VpLwPDY5KqH8hsDG45EXAMPLE
Default region name [None]: us-east-2
Default output format [None]: yaml
```

On the command line, invoke the `aws configure` command. You'll be prompted to enter the access key ID and secret access key from Figure 14-3.

You can also specify a default region and the default output format. The *region* is the geographic endpoint for your services. In this example, I'm telling AWS I want my services to use the `us-east-2` endpoint by default, which is in Ohio. You can find a general list of regional endpoints at *https://docs.aws.amazon.com/general/latest/gr/rande.html*.

Creating a Role

Your code requires a specific identity to run in AWS. This identity is called a *role*. You can have multiple roles under your AWS account and assign various permissions to each role. You can then assign roles to AWS services, which gives services permissions to access your resources without you having to assign credentials (such as your access key ID and secret access key) to each service. In this chapter, you'll use a role to give AWS Lambda permission to access the Lambda function you'll write.

For now, you'll create just one role and give AWS Lambda permission to assume that role so it can invoke your code. Listing 14-3 details a simple trust policy document that assigns the proper access. The trust policy document outlines a set of permissions, which you'll assign to a new role.

```
{
  "Version": ❶"2012-10-17",
  "Statement": [
    {
      "Effect": ❷"Allow",
      "Principal": {
        "Service": ❸"lambda.amazonaws.com"
      },
      "Action": ❹"sts:AssumeRole"
    }
  ]
}
```

*Listing 14-3: Defining a trust policy for your new role (*aws/trust-policy.json*)*

This trust policy tells AWS that you want to allow ❷ the Lambda service ❸ to assume the role ❹. The trust policy version ❶ is the current version of the trust policy language, not an arbitrary date.

Next, create the role to which you'll assign this trust policy:

```
$ aws iam create-role --role-name "lambda-xkcd" \
--assume-role-policy-document file://aws/trust-policy.json
Role:
❶ Arn: arn:aws:iam::123456789012:role/lambda-xkcd
❷ AssumeRolePolicyDocument:
    Statement:
    - Action: sts:AssumeRole
      Effect: Allow
      Principal:
        Service: lambda.amazonaws.com
    Version: '2012-10-17'
  CreateDate: '2006-01-02T15:04:05+00:00'
  Path: /
  RoleId: AROA1111111111EXAMPLE
  RoleName: lambda-xkcd
```

Create a new role with the AWS Identity and Access Management (IAM) service using the role name *lambda-xkcd* and the *aws/trust-policy.json* document you created in Listing 14-3. If successful, this creates a new role using your trust policy ❷. IAM assigns the role an Amazon Resource Name (ARN). The ARN ❶ is a unique identifier for this role that you'll use when invoking your code.

Defining an AWS Lambda Function

AWS Lambda's Go library gives you some flexibility when it comes to your Lambda function's signature. Your function must conform to one of these formats:

```
func()

func() error

func(TypeIn) error

func() (TypeOut, error)

func(context.Context) error

func(context.Context, TypeIn) error

func(context.Context) (TypeOut, error)

func(context.Context, TypeIn) (TypeOut, error)
```

`TypeIn` and `TypeOut` correspond to *encoding/json*-compatible types, in that JSON input sent to your Lambda function will be unmarshaled into `TypeIn`. Likewise, the `TypeOut` your function returns will be marshaled to JSON before reaching its destination. You'll use the last function signature in this section.

The function you'll write should give you a taste for what you can do with serverless environments. It will accept input from the client, retrieve resources over the internet, maintain its state between function calls, and respond to the client. If you've read Chapter 9, you know that you could write an `http.Handler` that performs these actions, but AWS Lambda requires a slightly different approach. You won't work with an `http.Request` or an `http.ResponseWriter`. Instead, you'll use types you create or import from other modules. AWS Lambda handles the decoding and encoding of the data to and from your function for you.

Let's get started writing your first bit of serverless code (Listing 14-4).

```
package main

import (
    "context"

    "github.com/awoodbeck/gnp/ch14/feed"
    "github.com/aws/aws-lambda-go/lambda"
)

var (
    rssFeed ❶feed.RSS
    feedURL = ❷"https://xkcd.com/rss.xml"
)

type EventRequest struct {
    Previous bool `json:"previous"`
}

type EventResponse struct {
    Title     string `json:"title"`
    URL       string `json:"url"`
    Published string `json:"published"`
}
```

*Listing 14-4: Creating persistent variables and request and response types (*aws/xkcd.go*)*

You can specify variables at the package level that will persist between function calls while the function persists in memory. In this example, you define a feed object ❶ and the URL of the RSS feed ❷. Creating and populating a new `feed.RSS` object involves a bit of overhead. You can avoid that overhead on subsequent function calls if you store the object in a variable at the package level so it lives beyond each function call. This also allows you to take advantage of the entity tag support in `feed.RSS`.

The `EventRequest` and `EventResponse` types define the format of a client request and the function's response. AWS Lambda unmarshals the JSON from the client's HTTP request body into the `EventRequest` object and marshals the function's `EventResponse` to JSON to the HTTP response body before returning it to the client.

Listing 14-5 defines the `main` function and begins to define the AWS Lambda-compatible function.

```
--snip--
func main() {
 ❶ lambda.Start(LatestXKCD)
}

func LatestXKCD(ctx context.Context, req EventRequest) (
    EventResponse, error) {
    resp := ❷EventResponse{Title: "xkcd.com", URL: "https://xkcd.com/"}

    if err := ❸rssFeed.ParseURL(ctx, feedURL); err != nil {
        return resp, err
    }
```

Listing 14-5: Main function and first part of the Lambda function named LatestXKCD (aws/xkcd.go)

Hook your function into Lambda by passing it to the `lambda.Start` method ❶. You're welcome to instantiate dependencies in an `init` function, or before this statement, if your function requires it.

The `LatestXKCD` function accepts a context and an `EventRequest` and returns an `EventResponse` and an `error` interface. It defines a response object ❷ with default `Title` and `URL` values. The function returns the response as is in the event of an error or an empty feed.

Parsing the feed URL ❸ from Listing 14-4 populates the `rssFeed` object with the latest feed details. Listing 14-6 uses these details to formulate the response.

```
--snip--
    switch items := rssFeed.Items(); {
    case ❶req.Previous && len(items) > 1:
        resp.Title = items[1].Title
        resp.URL = items[1].URL
        resp.Published = items[1].Published
    case len(items) > 0:
        resp.Title = items[0].Title
        resp.URL = items[0].URL
        resp.Published = items[0].Published
    }

    return resp, nil
}
```

Listing 14-6: Populating the response with the feed results (aws/xkcd.go)

If the client requests the previous XKCD comic ❶ and there are at least two feed items, the function populates the response with details of the previous XKCD comic. Otherwise, the function populates the response with the most recent XKCD comic details, provided there's at least one feed item. If neither of those cases is true, the client receives the response with its default values from Listing 14-5.

Compiling, Packaging, and Deploying Your Function

AWS Lambda expects you to compile your code and zip the resulting binary before deploying the archive, using the AWS CLI tools. To do this, use the following commands in Linux, macOS, or WSL:

```
$ GOOS=linux go build aws/xkcd.go
$ zip xkcd.zip xkcd
  adding: xkcd (deflated 50%)
$ aws lambda create-function --function-name "xkcd" --runtime "go1.x" \
--handler "xkcd" --role "arn:aws:iam::123456789012:role/lambda-xkcd" \
--zip-file "fileb://xkcd.zip"
CodeSha256: M36I7oiS8+S9AryIthcizsjdLDKXMaJKvZvsZzZDNHO=
CodeSize: 6597490
Description: ''
FunctionArn: arn:aws:lambda:us-east-2:123456789012:function:xkcd
FunctionName: ❶xkcd
Handler: ❷xkcd
LastModified: 2006-01-02T15:04:05.000+0000
LastUpdateStatus: Successful
MemorySize: 128
RevisionId: b094a881-9c49-4b86-86d5-eb4335507eb0
Role: arn:aws:iam::123456789012:role/lambda-xkcd
Runtime: go1.x
State: Active
Timeout: 3
TracingConfig:
  Mode: PassThrough
Version: $LATEST
```

Compile *aws/xkcd.go* and add the resulting *xkcd* binary to a ZIP file. Then, use the AWS CLI to create a new function named `xkcd`, a handler named `xkcd`, the `go1.x` runtime, the role ARN you created earlier, and the ZIP file containing the *xkcd* binary. Notice the *fileb://xkcd.zip* URL in the command line. This tells the AWS CLI that it can find a binary file (*fileb*) in the current directory named *xkcd.zip*.

If successful, the AWS CLI outputs the details of the new Lambda function: the function name ❶ in AWS, which you'll use on the command line to manage your function, and the filename of the binary in the zip file ❷.

Compilation of the binary and packing it is a bit different on Windows. I recommend you do this in PowerShell since you can compress the cross-compiled binary on the command line without the need to install a specific archiver.

```
PS C:\Users\User\dev\gnp\ch14> setx GOOS linux

SUCCESS: Specified value was saved.
PS C:\Users\User\dev\gnp\ch14> \Go\bin\go.exe build -o xkcd .\aws\xkcd.go
go: downloading github.com/aws/aws-lambda-go v1.19.1
--snip--
PS C:\Users\User\dev\gnp\ch14> Compress-Archive xkcd xkcd.zip
```

At this point, use the AWS CLI tools to deploy the ZIP file as in the previous listing.

If you need to update your function code, recompile the binary and archive it again. Then use the following command to update the existing Lambda function:

```
$ aws lambda update-function-code --function-name "xkcd" \
--zip-file "fileb://xkcd.zip"
```

Since you're updating an existing function, the only values you need to provide are the names of the function and ZIP file. AWS takes care of the rest.

As an exercise, update the code to allow the client to request a forced refresh of the XKCD RSS feed. Then, update the function with those changes and move on to the next section to test those changes.

Testing Your AWS Lambda Function

The AWS CLI tools make it easy to test your Lambda function. You can use them to send a JSON payload and capture the JSON response. Invoke the function by providing the function name and the path to a file in the AWS CLI. The AWS CLI will populate this with the response body:

```
$ aws lambda invoke --function-name "xkcd" response.json
ExecutedVersion: $LATEST
StatusCode: 200
```

If the invocation is successful, you can verify that your function provided the XKCD comic name and URL by reviewing the *response.json* contents:

```
$ cat response.json
{"title":"Election Screen Time","url":"https://xkcd.com/2371/",
"published":"Mon, 12 Oct 2020 04:00:00 -0000"}
```

You can also invoke the function with a custom request body by adding a few additional command line arguments. You can pass a `payload` string if you specify its format as `raw-in-base64-out`. This tells the AWS CLI to take the string you provide and Base64-encode it before assigning it to the request body and passing it along to the function:

```
$ aws lambda invoke --cli-binary-format "raw-in-base64-out" \
--payload '{"previous":true}' --function-name "xkcd" response.json
ExecutedVersion: $LATEST
StatusCode: 200
$ cat response.json
{"title":"Chemist Eggs","url":"https://xkcd.com/2373/",
"published":"Fri, 16 Oct 2020 04:00:00 -0000"}
```

Google Cloud Functions

Like AWS Lambda, Google Cloud Functions allows you to deploy code in a serverless environment, offloading the implementation details to Google. Not surprisingly, Go enjoys first-class support in Cloud Functions.

You'll need a Google Cloud account before proceeding with this section. Visit *https://cloud.google.com* to get started with a trial account.

Installing the Google Cloud Software Development Kit

The Google Cloud Software Development Kit (SDK) requires Python 2.7.9 or 3.5+. You'll need to make sure a suitable version of Python is installed on your operating system before proceeding. You can follow Google's comprehensive installation guide at *https://cloud.google.com/sdk/docs/install/*, where you'll find specific installation instructions for Windows, macOS, and various flavors of Linux.

Here are the generic Linux installation steps:

```
$ curl -O https://dl.google.com/dl/cloudsdk/channels/rapid/downloads/\
google-cloud-sdk-319.0.1-linux-x86_64.tar.gz
  % Total    % Received % Xferd  Average Speed   Time    Time     Time
Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 81.9M  100 81.9M    0     0  34.1M      0  0:00:02  0:00:02 --:--:-- 34.1M
$ tar xf google-cloud-sdk-319.0.1-linux-x86_64.tar.gz
$ ./google-cloud-sdk/install.sh
Welcome to the Google Cloud SDK!
--snip--
```

Download the current Google Cloud SDK tarball (the version changes frequently!) and extract it. Then, run the *./google-cloud-sdk/install.sh* script. The installation process asks you questions that pertain to your environment. I snipped them from the output for brevity.

Initializing the Google Cloud SDK

You need to authorize the Google Cloud SDK before you're able to use it to deploy your code. Google makes this process simple compared to AWS. There's no need to create credentials and then copy and paste them to the command line. Instead, Google Cloud uses your web browser for authentication and authorization.

The `gcloud init` command is equivalent to the `aws configure` command, in that it will walk you through the configuration of your Google Cloud command line environment:

```
$ ./google-cloud-sdk/bin/gcloud init
Welcome! This command will take you through the configuration of gcloud.
--snip--
Pick cloud project to use:
```

```
❶ [1] Create a new project
  Please enter numeric choice or text value (must exactly match list
  item):  1

  Enter a Project ID. Note that a Project ID CANNOT be changed later.
  Project IDs must be 6-30 characters (lowercase ASCII, digits, or
  hyphens) in length and start with a lowercase letter. goxkcd
  --snip--
  $ gcloud projects list
  PROJECT_ID                      NAME                    PROJECT_NUMBER
  goxkcd                          goxkcd                  123456789012
```

The first step in the process will open a page in your web browser to authenticate your Google Cloud SDK with your Google Cloud account. Your command line output may look a little different from the output here if your Google Cloud account has existing projects. For the purposes of this chapter, elect to create a new project ❶ and give it a project ID—`goxkcd` in this example. Your project ID must be unique across Google Cloud. Once you've completed this step, you're ready to interact with Google Cloud from the command line, just as you did with AWS.

Enable Billing and Cloud Functions

You need to make sure billing is enabled for your project before it can use Cloud Functions. Visit *https://cloud.google.com/billing/docs/how-to/modify-project/* to learn how to modify the billing details of an existing project. Once enabled, you can then enable your project's Cloud Functions access. At this point, you can start writing code.

Defining a Cloud Function

Cloud Functions uses Go's module support instead of requiring you to write a stand-alone application as you did for AWS Lambda. This simplifies your code a bit since you don't need to import any libraries specific to Cloud Functions or define a main function as the entry point of execution.

Listing 14-7 provides the initial code for a Cloud Functions–compatible module.

```
package gcp

import (
    "encoding/json"
    "log"
    "net/http"

    "github.com/awoodbeck/gnp/ch14/feed"
)

var (
    rssFeed feed.RSS
    feedURL = "https://xkcd.com/rss.xml"
)
```

```
type EventRequest struct {
    Previous bool `json:"previous"`
}

type EventResponse struct {
    Title          string `json:"title"`
    URL            string `json:"url"`
    Published      string `json:"published"`
}
```

*Listing 14-7: Creating persistent variables and request and response types (*gcp/xkcd.go*)*

The types are identical to the code we wrote for AWS Lambda. Unlike AWS Lambda, Cloud Functions won't unmarshal the request body into an `EventRequest` for you. Therefore, you'll have to handle the unmarshaling and marshaling of the request and response payloads on your own.

Whereas AWS Lambda accepted a range of function signatures, Cloud Functions uses the familiar `net/http` handler function signature: `func(http.ResponseWriter, *http.Request)`, as shown in Listing 14-8.

```
--snip--
func LatestXKCD(w http.ResponseWriter, r *http.Request) {
    var req EventRequest
    resp := EventResponse{Title: "xkcd.com", URL: "https://xkcd.com/"}

    defer ❶func() {
        w.Header().Set("Content-Type", "application/json")
        out, _ := json.Marshal(&resp)
        _, _ = w.Write(out)
    }()

    if err := ❷json.NewDecoder(r.Body).Decode(&req); err != nil {
        log.Printf("decoding request: %v", err)
        return
    }

    if err := rssFeed.ParseURL(❸r.Context(), feedURL); err != nil {
        log.Printf("parsing feed: %v:", err)
        return
    }
```

*Listing 14-8: Handling the request and response and optionally updating the RSS feed (*gcp/xkcd.go*)*

Like the AWS code, this `LatestXKCD` function refreshes the RSS feed by using the `ParseURL` method. But unlike the equivalent AWS code, you need to JSON-unmarshal the request body ❷ and marshal the response to JSON ❶ before sending it to the client. Even though `LatestXKCD` doesn't receive a context in its function parameters, you can use the request's context ❸ to cancel the parser if the socket connection with the client terminates before the parser returns.

Listing 14-9 implements the remainder of the `LatestXKCD` function.

```
--snip--
    switch items := rssFeed.Items(); {
    case req.Previous && len(items) > 1:
        resp.Title = items[1].Title
        resp.URL = items[1].URL
        resp.Published = items[1].Published
    case len(items) > 0:
        resp.Title = items[0].Title
        resp.URL = items[0].URL
        resp.Published = items[0].Published
    }
}
```

*Listing 14-9: Populating the response with the feed results (*gcp/xkcd.go*)*

Like Listing 14-6, this code populates the response fields with the appropriate feed item. The deferred function in Listing 14-8 handles writing the response to the `http.ResponseWriter`, so there's nothing further to do here.

Deploying Your Cloud Function

You need to address one bit of module accounting before you deploy your code; you need to create a *go.mod* file so Google can find dependencies, because unlike with AWS Lambda, you don't compile and package the binary yourself. Instead, the code is ultimately compiled on Cloud Functions.

Use the following commands to create the *go.mod* file:

```
$ cd gcp
gcp$ go mod init github.com/awoodbeck/gnp/ch14/gcp
go: creating new go.mod: module github.com/awoodbeck/gnp/ch14/gcp
gcp$ go mod tidy
--snip--
gcp$ cd -
```

These commands initialize a new module named *github.com/awoodbeck/gnp/ch14/gcp* and tidy the module requirements in the *go.mod* file.

Your module is ready for deployment. Use the `gcloud functions deploy` command, which accepts your code's function name, the source location, and the Go runtime version:

```
$ gcloud functions deploy LatestXKCD --source ./gcp/ --runtime go113 \
--trigger-http --allow-unauthenticated
Deploying function (may take a while - up to 2 minutes)...
For Cloud Build Stackdriver Logs, visit:
https://console.cloud.google.com/logs/viewer--snip--
Deploying function (may take a while - up to 2 minutes)...done.
availableMemoryMb: 256
buildId: 5d7fee9b-7468-4b04-badc-81015aa62e59
entryPoint: ❶LatestXKCD
httpsTrigger:
  url: ❷https://us-central1-goxkcd.cloudfunctions.net/LatestXKCD
ingressSettings: ❸ALLOW_ALL
```

```
labels:
  deployment-tool: cli-gcloud
name: projects/goxkcd/locations/us-central1/functions/LatestXKCD
runtime: ❹go113
serviceAccountEmail: goxkcd@appspot.gserviceaccount.com
sourceUploadUrl: https://storage.googleapis.com/--snip--
status: ACTIVE
timeout: 60s
updateTime: '2006-01-02T15:04:05.000Z'
versionId: '1'
```

The addition of the `--trigger-http` and `--allow-unauthenticated` flags tells Google you want to trigger a call to your function by an incoming HTTP request and that no authentication is required for the HTTP endpoint.

Once created, the SDK output shows the function name ❶, the HTTP endpoint ❷ for your function, the permissions for the endpoint ❸, and the Go runtime version ❹.

Although the Cloud Functions deployment workflow is simpler than the AWS Lambda workflow, there's a limitation: you're restricted to the Go runtime version that Cloud Functions supports, which may not be the latest version. Therefore, you need to make sure the code you write doesn't use newer features added since Go 1.13. You don't have a similar limitation when deploying to AWS Lambda, since you locally compile the binary before deployment.

Testing Your Google Cloud Function

The Google Cloud SDK doesn't include a way to invoke your function from the command line, as you did using the AWS CLI. But your function's HTTP endpoint is publicly accessible, so you can directly send HTTP requests to it.

Use `curl` to send HTTP requests to your function's HTTP endpoint:

```
$ curl -X POST -H "Content-Type: application/json" --data '{}' \
https://us-central1-goxkcd.cloudfunctions.net/LatestXKCD
{"title":"Chemist Eggs","url":"https://xkcd.com/2373/",
"published":"Fri, 16 Oct 2020 04:00:00 -0000"}
$ curl -X POST -H "Content-Type: application/json" \
--data '{"previous":true}' \
https://us-central1-goxkcd.cloudfunctions.net/LatestXKCD
{"title":"Chemist Eggs","url":"https://xkcd.com/2373/",
"published":"Fri, 16 Oct 2020 04:00:00 -0000"}
```

Here, you send `POST` requests with the `Content-Type` header indicating that the request body contains JSON. The first request sends an empty object, so you correctly receive the current XKCD comic title and URL. The second request asks for the previous comic, which the function correctly returns in its response.

Keep in mind that, unlike with AWS, the only security your function's HTTP endpoint has with the use of the `--allow-unauthenticated` flag is obscurity, as anyone can send requests to your Google Clouds function. Since you

aren't returning sensitive information, the main risk you face is the potential cost you may incur if you neglect to delete or secure your function after you're done with it.

Once you're satisfied that the function works as expected, go ahead and delete it. I'll sleep better at night if you do. You can remove the function from the command line like this:

```
$ gcloud functions delete LatestXKCD
```

You'll be prompted to confirm the deletion.

Azure Functions

Unlike AWS Lambda and Google Cloud Functions, Microsoft Azure Functions doesn't offer first-class support for Go. But all is not lost. We can define a custom handler that exposes an HTTP server. Azure Functions will proxy requests and responses between clients and your custom handler's HTTP server. You can read more details about the Azure Functions custom handlers at *https://docs.microsoft.com/en-us/azure/azure-functions/functions-custom-handlers#http-only-function*. In addition, your code runs in a Windows environment as opposed to Linux, which is an important distinction when compiling your code for deployment on Azure Functions.

You'll need a Microsoft Azure account before proceeding. Visit *https://azure.microsoft.com* to create one.

Installing the Azure Command Line Interface

The Azure CLI has installation packages for Windows, macOS, and several popular Linux distributions. You can find details for your operating system at *https://docs.microsoft.com/en-us/cli/azure/install-azure-cli/*.

The following commands install the Azure CLI on a Debian-compatible Linux system:

```
$ curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
[sudo] password for user:
export DEBIAN_FRONTEND=noninteractive
apt-get update
--snip--
$ az version
{
  "azure-cli": "2.15.0",
  "azure-cli-core": "2.15.0",
  "azure-cli-telemetry": "1.0.6",
  "extensions": {}
}
```

The first command downloads the `InstallAzureCLIDeb` shell script and pipes it to `sudo bash`. After authenticating, the script installs an Apt repository, updates Apt, and installs the *azure-cli* package.

Once installed, the `az version` command displays the current Azure CLI component versions.

Configuring the Azure CLI

Whereas the AWS CLI required you to provide its credentials during configuration, and the Google Cloud SDK opened a web page to authorize itself during configuration, the Azure CLI separates configuration and authentication into separate steps. First, issue the `az configure` command and follow the instructions for configuring the Azure CLI. Then, run the `az login` command to authenticate your Azure CLI using your web browser:

```
$ az configure
Welcome to the Azure CLI! This command will guide you through logging in and
setting some default values.

Your settings can be found at /home/user/.azure/config
Your current configuration is as follows:
--snip--
$ az login
❶ You have logged in. Now let us find all the subscriptions to which you have
access...
[
  {
    "cloudName": "AzureCloud",
--snip--
  }
]
```

The Azure CLI supports several configuration options not covered in the `az configure` process. You can use the Azure CLI to set these values instead of directly editing the *$HOME/.azure/config* file. For example, you can disable telemetry by setting the `core.collect_telemetry` variable to `off`:

```
$ az config set core.collect_telemetry=off
Command group 'config' is experimental and not covered by customer support.
Please use with discretion.
```

Installing Azure Functions Core Tools

Unlike the other cloud services covered in this chapter, the Azure CLI tools do not directly support Azure Functions. You need to install another set of tools specific to Azure Functions.

The "Install the Azure Functions Core Tools" section of *https://docs.microsoft.com/en-us/azure/azure-functions/functions-run-local/* details the process of installing version 3 of the tools on Windows, macOS, and Linux.

Creating a Custom Handler

You can use the Azure Functions core tools to initialize a new custom handler. Simply run the `func init` command, setting the `--worker-runtime` flag to `custom`:

```
$ cd azure
$ func init --worker-runtime custom
Writing .gitignore
Writing host.json
Writing local.settings.json
Writing /home/user/dev/gnp/ch14/azure/.vscode/extensions.json
```

The core tools then create a few project files, the most relevant to us being the *host.json* file.

You need to complete a few more tasks before you start writing code. First, create a subdirectory named after your desired function name in Azure Functions:

```
$ mkdir LatestXKCDFunction
```

This example names the Azure Function `LatestXKCDFunction` by creating a subdirectory with the same name. This name will be part of your function's endpoint URL.

Second, create a file named *function.json* in the subdirectory with the contents in Listing 14-10.

```
{
  "bindings": [
    {
      "type": ❶"httpTrigger",
      "direction": ❷"in",
      "name": "req",
   ❸  "methods": [ "post" ]
    },
    {
      "type": ❹"http",
      "direction": ❺"out",
      "name": "res"
    }
  ]
}
```

*Listing 14-10: Binds incoming HTTP trigger and outgoing HTTP (*azure/LatestXKCDFunction/function.json*)*

The Azure Functions Core Tools will use this *function.json* file to configure Azure Functions to use your custom handler. This JSON instructs Azure Functions to bind an incoming HTTP trigger to your custom handler and expect HTTP output from it. Here, you're telling Azure Functions that incoming ❷ `POST` requests ❸ shall trigger ❶ your custom handler, and your custom handler returns ❹ HTTP responses ❺.

Lastly, the generated *host.json* file needs some tweaking (Listing 14-11).

```
{
  "version": "2.0",
  "logging": {
    "applicationInsights": {
      "samplingSettings": {
        "isEnabled": true,
        "excludedTypes": "Request"
      }
    }
  },
  "extensionBundle": {
    "id": "Microsoft.Azure.Functions.ExtensionBundle",
    "version": "[1.*, 2.0.0)"
  },
  "customHandler": {
❶ "enableForwardingHttpRequest": true,
    "description": {
      "defaultExecutablePath": ❷"xkcd.exe",
      "workingDirectory": "",
      "arguments": []
    }
  }
}
```

*Listing 14-11: Tweaking the host.json file (*azure/host.json*)*

Make sure to enable the forwarding of HTTP requests from Azure Functions to your custom handler ❶. This instructs Azure Functions to act as a proxy between clients and your custom handler. Also, set the default executable path to the name of your Go binary ❷. Since your code will run on Windows, make sure to include the *.exe* file extension.

Defining a Custom Handler

Your custom handler needs to instantiate its own HTTP server, but you can leverage code you've already written for Google Cloud Functions. Listing 14-12 is the entire custom handler implementation.

```
package main

import (
    "log"
    "net/http"
    "os"
    "time"

    "github.com/awoodbeck/gnp/ch14/gcp"
)

func main() {
    port, exists := ❶os.LookupEnv("FUNCTIONS_CUSTOMHANDLER_PORT")
    if !exists {
        log.Fatal("FUNCTIONS_CUSTOMHANDLER_PORT environment variable not set")
    }
```

```
    srv := &http.Server{
        Addr:              ":" + port,
        Handler:           http.HandlerFunc(❷gcp.LatestXKCD),
        IdleTimeout:       time.Minute,
        ReadHeaderTimeout: 30 * time.Second,
    }

    log.Printf("Listening on %q ...\n", srv.Addr)
    log.Fatal(srv.ListenAndServe())
}
```

*Listing 14-12: Using the Google Cloud Functions code to handle requests (*azure/xkcd.go*)*

Azure Functions expects your HTTP server to listen to the port number it assigns to the `FUNCTIONS_CUSTOMHANDLER_PORT` environment variable ❶. Since the `LatestXKCD` function you wrote for Cloud Functions can be cast as an `http.HandlerFunc`, you can save a bunch of keystrokes by importing its module and using the function as your HTTP server's handler ❷.

Locally Testing the Custom Handler

The Azure Functions Core Tools allow you to locally test your code before deployment. Let's walk through the process of building and running the Azure Functions code on your computer. First, change into the directory with your Azure Functions code:

```
$ cd azure
```

Next, build your code, making sure that the resulting binary name matches the one you defined in your host file—*xkcd.exe*, in this example:

```
azure$ go build -o xkcd.exe xkcd.go
```

Since your code will run locally, you do not need to explicitly compile your binary for Windows.

Finally, run `func start`, which will read the *host.json* file and execute the *xkcd.exe* binary:

```
azure$ func start
Azure Functions Core Tools (3.0.2931 Commit hash:
d552c6741a37422684f0efab41d541ebad2b2bd2)
Function Runtime Version: 3.0.14492.0
[2020-10-18T16:07:21.857] Worker process started and initialized.
[2020-10-18T16:07:21.915] 2020/10/18 12:07:21 Listening on ❶":44687" ...
[2020-10-18T16:07:21.915] 2020/10/18 12:07:21 decoding request: EOF
Hosting environment: Production
Content root path: /home/user/dev/gnp/ch14/azure
Now listening on: ❷http://0.0.0.0:7071
Application started. Press Ctrl+C to shut down.
```

```
Functions:

    LatestXKCDFunction: [POST] ❸http://localhost:7071/api/LatestXKCDFunction

For detailed output, run func with -verbose flag.
```

Here, the Azure Functions code set the `FUNCTIONS_CUSTOMHANDLER_PORT` environment variable to 44687 ❶ before executing the *xkcd.exe* binary. Azure Functions also exposes an HTTP endpoint on port 7071 ❷. Any requests sent to the *LatestXKCDFunction* endpoint ❸ are forwarded onto the *xkcd.exe* HTTP server, and responses are forwarded to the client.

Now that the *LatestXKCDFunction* endpoint is active, you can send HTTP requests to it as you did with your Google Cloud Functions code:

```
$ curl -X POST -H "Content-Type: application/json" --data '{}' \
http://localhost:7071/api/LatestXKCDFunction
{"title":"Chemist Eggs","url":"https://xkcd.com/2373/",
"published":"Fri, 16 Oct 2020 04:00:00 -0000"}
$ curl -X POST -H "Content-Type: application/json" -data \
'{"previous":true}' http://localhost:7071/api/LatestXKCDFunction
{"title":"Dialect Quiz","url":"https://xkcd.com/2372/",
"published":"Wed, 14 Oct 2020 04:00:00 -0000"}
```

As with Google Cloud, sending a `POST` request with empty JSON in the request body causes the custom handler to return the current XKCD comic title and URL. Requesting the previous comic accurately returns the previous comic's title and URL.

Deploying the Custom Handler

Since you're using a custom handler, the deployment process is slightly more complicated than that for Lambda or Cloud Functions. This section walks you through the steps on Linux. You can find the entire process detailed at *https://docs.microsoft.com/en-us/azure/azure-functions/functions-create-first-azure-function-azure-cli/*.

Start by issuing the `az login` command to make sure your Azure CLI's authorization is current:

```
$ az login
You have logged in.
```

Next, create a resource group and specify the location you'd like to use. You can get a list of locations using `az account list-locations`. This example uses `NetworkProgrammingWithGo` for the resource group name and `eastus` for the location:

```
$ az group create --name NetworkProgrammingWithGo --location eastus
{
  "id": "/subscriptions/--snip--/resourceGroups/NetworkProgrammingWithGo",
  "location": "eastus",
```

```
  "managedBy": null,
  "name": "NetworkProgrammingWithGo",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
```

Then, create a unique storage account, specifying its name, location, the resource group name you just created, and the `Standard_LRS` SKU:

```
$ az storage account create --name npwgstorage --location eastus \
--resource-group NetworkProgrammingWithGo --sku Standard_LRS
 - Finished ..
--snip--
```

Finally, create a function application with a unique name, making sure to specify you're using Functions 3.0 and a custom runtime:

```
$ az functionapp create --resource-group NetworkProgrammingWithGo \
--consumption-plan-location eastus --runtime custom \
--functions-version 3 --storage-account npwgstorage --name latestxkcd
Application Insights "latestxkcd" was created for this Function App.
--snip--
```

At this point, you're ready to compile your code and deploy it. Since your code will run on Windows, it's necessary to build your binary for Windows. Then, publish your custom handler.

```
$ cd azure
azure$ GOOS=windows go build -o xkcd.exe xkcd.go
azure$ func azure functionapp publish latestxkcd --no-build
Getting site publishing info...
Creating archive for current directory…
Skipping build event for functions project (--no-build).
Uploading 6.12 MB [############################################################]
Upload completed successfully.
Deployment completed successfully.
Syncing triggers...
Functions in latestxkcd:
    LatestXKCDFunction - [httpTrigger]
        Invoke url: ❶https://latestxkcd.azurewebsites.net/api/
latestxkcdfunction
```

Once the code is deployed, you can send `POST` requests to your custom handler's URL ❶. The actual URL is a bit longer than this one, and it includes URI parameters relevant to Azure Functions. I've snipped it for brevity.

Testing the Custom Handler

Assuming you're using your custom handler's full URL, it should return results like those seen here:

```
$ curl -X POST -H "Content-Type: application/json" --data '{}' \
https://latestxkcd.azurewebsites.net/api/latestxkcdfunction?--snip--
{"title":"Chemist Eggs","url":"https://xkcd.com/2373/",
"published":"Fri, 16 Oct 2020 04:00:00 -0000"}
$ curl -X POST -H "Content-Type: application/json" \
--data '{"previous":true}' \
https://latestxkcd.azurewebsites.net/api/latestxkcdfunction?--snip--
{"title":"Chemist Eggs","url":"https://xkcd.com/2373/",
"published":"Fri, 16 Oct 2020 04:00:00 -0000"}
```

Use `curl` to query your Azure Functions custom handler. As expected, empty JSON results in the current XKCD comic's title and URL, whereas a request for the previous comic properly returns the previous comic's details.

What You've Learned

When you use cloud offerings, you can focus on application development and avoid the costs of acquiring a server infrastructure, software licensing, and the human resources required to maintain it all. This chapter explored Amazon Web Services, Google Cloud, and Microsoft Azure, all of which offer comprehensive solutions that allow you to scale your business and pay as you go. We used AWS Lambda, Google Cloud Functions, and Microsoft Azure Functions, which are all PaaS offerings that allow you to deploy an application while letting the platform handle the implementation details.

As you saw, developing and deploying an application on the three cloud environments follow the same general process. First, you install the platform's command line tools. Next, you authorize the command line tools to act on behalf of your account. You then develop your application for the target platform and deploy it. Finally, you make sure your application works as expected.

Both AWS Lambda and Cloud Functions have first-class support for Go, making the development and deployment workflow easy. Although Azure Functions doesn't explicitly support Go, you can write a custom handler to use with the service. But despite the small variations in the development, deployment, and testing workflows, all three cloud platforms can generate the same result. Which one you should use comes down to your use case and budget.

INDEX

D

E

F

G

H

I

J

K

L

M

N

O

P

R

S

T

U

W

X

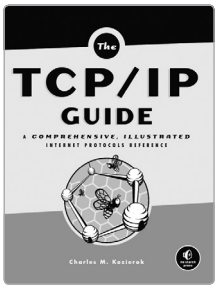
Z

Network Programming with Go is set in New Baskerville, Futura, Dogma, and TheSansMono Condensed.

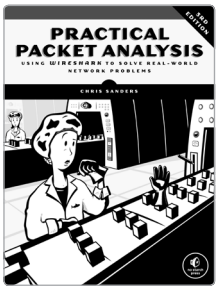
RESOURCES

Visit *https://nostarch.com/networkprogrammingwithgo/* for errata and more information.

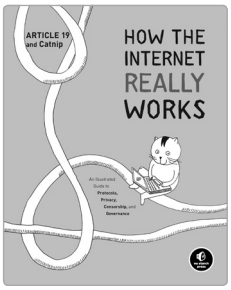
More no-nonsense books from **NO STARCH PRESS**



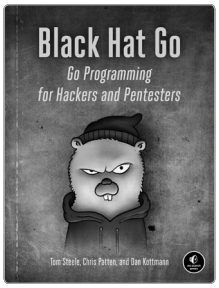
THE TCP/IP GUIDE
A Comprehensive, Illustrated Internet Protocols Reference
BY CHARLES M. KOZIEROK
1616 PP., $99.95
ISBN 978-1-59327-047-6



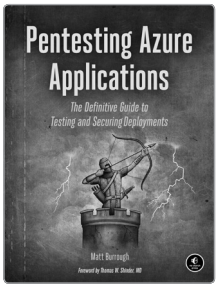
PRACTICAL PACKET ANALYSIS 3RD EDITION
Using Wireshark to Solve Real-World Network Problems
BY CHRIS SANDERS
368 PP., $49.95
ISBN 978-1-59327-802-1



HOW THE INTERNET REALLY WORKS
An Illustrated Guide to Protocols, Privacy, Censorship, and Governance
BY ARTICLE 19
120 PP., $19.95
ISBN 978-1-71850-029-7



BLACK HAT GO
Go Programming for Hackers and Pentesters
BY TOM STEELE, CHRIS PATTEN, *AND* DAN KOTTMANN
368 PP., $39.95
ISBN 978-1-59327-865-6



PENTESTING AZURE APPLICATIONS
The Definitive Guide to Testing and Securing Deployments
BY MATT BURROUGH
216 PP., $39.95
ISBN 978-1-59327-863-2

THE RUST PROGRAMMING LANGUAGE
Covers Rust 2018
BY STEVE KLABNIK *AND* CAROL NICHOLS
560 PP., $39.95
ISBN 978-1-71850-044-0

PHONE:
800.420.7240 OR
415.863.9900

EMAIL:
SALES@NOSTARCH.COM

WEB:
WWW.NOSTARCH.COM

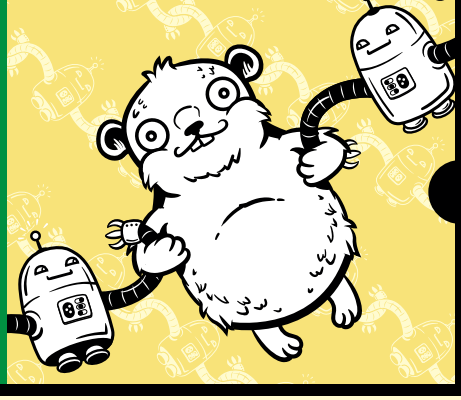
Never before has the world relied so heavily on the Internet to stay connected and informed. That makes the Electronic Frontier Foundation's mission—to ensure that technology supports freedom, justice, and innovation for all people—more urgent than ever.

For over 30 years, EFF has fought for tech users through activism, in the courts, and by developing software to overcome obstacles to your privacy, security, and free expression. This dedication empowers all of us through darkness. With your help we can navigate toward a brighter digital future.

ELECTRONIC FRONTIER FOUNDATION EFF

LEARN MORE AND JOIN EFF AT EFF.ORG/NO-STARCH-PRESS

COVERS GO 1.15 (BACKWARD COMPATIBLE WITH GO 1.12 AND HIGHER)



BUILD SIMPLE, RELIABLE NETWORK SOFTWARE

Combining the best parts of many other programming languages, Go is fast, scalable, and designed for high-performance networking and multiprocessing. In other words, it's perfect for network programming.

Network Programming with Go will help you leverage Go to write secure, readable, production-ready network code. In the early chapters, you'll learn the basics of networking and traffic routing. Then you'll put that knowledge to use as the book guides you through writing programs that communicate using TCP, UDP, and Unix sockets to ensure reliable data transmission.

As you progress, you'll explore higher-level network protocols like HTTP and HTTP/2 and build applications that securely interact with servers, clients, and APIs over a network using TLS.

You'll also learn:

- Internet Protocol basics, such as the structure of IPv4 and IPv6, multicasting, DNS, and network address translation
- Methods of ensuring reliability in socket-level communications
- Ways to use handlers, middleware, and multiplexers to build capable HTTP applications with minimal code
- Tools for incorporating authentication and encryption into your applications using TLS
- Methods to serialize data for storage or transmission in Go-friendly formats like JSON, Gob, XML, and protocol buffers
- Ways of instrumenting your code to provide metrics about requests, errors, and more
- Approaches for setting up your application to run in the cloud (and reasons why you might want to)

Networking Programming with Go is all you'll need to take advantage of Go's built-in concurrency, rapid compiling, and rich standard library.

ABOUT THE AUTHOR

Adam Woodbeck is a senior software engineer at Barracuda Networks, where he implemented a distributed cloud environment in Go. He has also served as the architect for many network-based services in Go.

no starch press

THE FINEST IN GEEK ENTERTAINMENT™

www.nostarch.com

$49.99 ($65.99 CDN)

ISBN 978-1-7185-0088-4

9 781718 500884 54999